

BAB 3

IMPLEMENTASI PROYEK

3.1. Profil Mitra

3.1.1. Nama Organisasi/Lembaga Mitra

Fakultas Ilmu Komputer Universitas Pembangunan Nasional Veteran Jakarta

3.1.2. Deskripsi Mitra

Berdiri sejak tahun 1995, Fakultas Ilmu Komputer Universitas Pembangunan Nasional Veteran Jakarta (UPNVJ) memiliki sejarah yang cukup signifikan. Pembentukan fakultas ini merupakan pengembangan lebih lanjut dari Program Pendidikan Ahli Teknik Informatika yang telah berdiri sejak tahun 1980. Sejak UPNVJ memperoleh status Perguruan Tinggi Negeri pada tahun 2014, fakultas ini secara terus menerus menunjukkan komitmen dalam menghasilkan lulusan yang kompeten di bidang teknologi informasi dan komunikasi, dengan semangat bela negara.

Sebagai salah satu institusi pendidikan tinggi yang dinamis, Fakultas Ilmu Komputer UPNVJ menyelenggarakan beragam program pendidikan, penelitian, serta pengabdian kepada masyarakat yang bersifat inovatif. Fokus utamanya adalah untuk menghasilkan sumber daya manusia yang memiliki kompetensi tinggi, mendukung perkembangan teknologi melalui kegiatan penelitian, serta memberikan kontribusi yang substansial kepada masyarakat luas. Selain itu, fakultas ini juga berupaya membangun lingkungan akademik yang kondusif melalui implementasi kurikulum yang responsif terhadap kebutuhan pemangku kepentingan dan memiliki orientasi terhadap kemajuan teknologi. Guna merealisasikan visi fakultas sebagai institusi yang berorientasi internasional, Fakultas Ilmu Komputer UPNVJ juga secara aktif menjalin kolaborasi dengan berbagai institusi pendidikan tinggi dan sektor industri, baik di tingkat nasional maupun internasional. Dengan tata kelola yang profesional dan berlandaskan nilai integritas, Fakultas Ilmu Komputer terus berkembang menjadi pusat inovasi terkemuka dalam bidang teknologi informasi dan komunikasi.

3.1.3. Hubungan Mitra dengan Proyek

Proses pengembangan dan implementasi proyek ini melibatkan beberapa mitra strategis guna mendukung keberhasilannya. Hal tersebut dikarenakan proyek yang dikembangkan memiliki tujuan untuk melakukan digitalisasi proses bimbingan tugas akhir antara mahasiswa dengan dosen pembimbing yang harapannya dapat meningkatkan efektivitas dalam prosesnya. Berikut merupakan keterkaitan antara mitra dengan proyek yang dikembangkan:

1. Fakultas Ilmu Komputer Universitas Pembangunan Nasional Veteran Jakarta

a. Peran

- i. Menjadi peran sentral dalam pelaksanaan pengembangan dan implementasi proyek.
- ii. Mengalokasikan sumber daya manusia, termasuk dosen dan mahasiswa, untuk proses perancangan, pengembangan, dan pengujian sistem.

b. Manfaat:

- i. Menjadi solusi inovatif yang dapat mengoptimalkan operasional dalam proses membimbing mahasiswa tingkat akhir yang sedang mengerjakan tugas akhir.
- ii. Berkontribusi terhadap peningkatan kualitas layanan operasional fakultas.

2. Dosen Pembimbing dan Mahasiswa Tingkat Akhir

a. Peran

- i. Sebagai pengguna utama dalam proyek yang dikembangkan dan diimplementasi khususnya pada proses bimbingan tugas akhir.

b. Manfaat:

- i. Meningkatkan kualitas dalam proses bimbingan oleh dosen pembimbing terhadap mahasiswa.

- ii. Memberikan kemudahan dalam proses pencatatan dan pemantauan kemajuan pengerjaan tugas akhir.

3.2. Metode Implementasi

Bagian ini menguraikan tahapan implementasi proyek pengembangan sistem bimbingan tugas akhir dengan menggunakan metode *Extreme Programming* (XP). Implementasi dibagi menjadi dua fokus utama yang berjalan secara paralel dan terintegrasi, yaitu pengembangan antarmuka pengguna (*front-end*) dan pengembangan logika pengelolaan data (*back-end*). Proses pengembangan *front-end* akan melibatkan perancangan dan implementasi tampilan yang interaktif oleh mahasiswa dan dosen pembimbing. Sementara pengembangan *back-end* berfokus pada pengembangan pada pengembangan basis data dan pengembangan *Application Programming Interface* (API) yang menghubungkan *front-end* dengan data dan fungsionalitas sistem secara keseluruhan.

3.2.1. Implementasi *Back-end*

a. *Separation of Concerns* (SoC)

Separation of Concerns (SoC) merupakan prinsip desain yang memfokuskan pada pemisahan tanggung jawab dalam sebuah kode program, yang dalam penerapannya setiap bagian kode memiliki tugas yang spesifik dan terdefinisi dengan baik (Lakshmi dan Chaitanya Josyula 2023). Menurut Lakshmi dan Chaitanya Josyula (2023) di dalam studinya, penerapan SoC membagi logika aplikasi menjadi modul-modul yang lebih kecil dan terkelola, seperti modul untuk manajemen data, autentikasi pengguna, atau pemrosesan transaksi. Pemisahan ini bertujuan untuk mengurangi kompleksitas kode, meningkatkan kemudahan dalam membaca kode, dan mempermudah pemeliharaan serta pengembangan fitur baru dengan memanfaatkan modul yang dapat digunakan berulang kali. Dengan mengisolasi setiap area fungsionalitas atau “*concern*” ke dalam modul terpisah, perubahan pada satu bagian dari sistem memiliki dampak yang

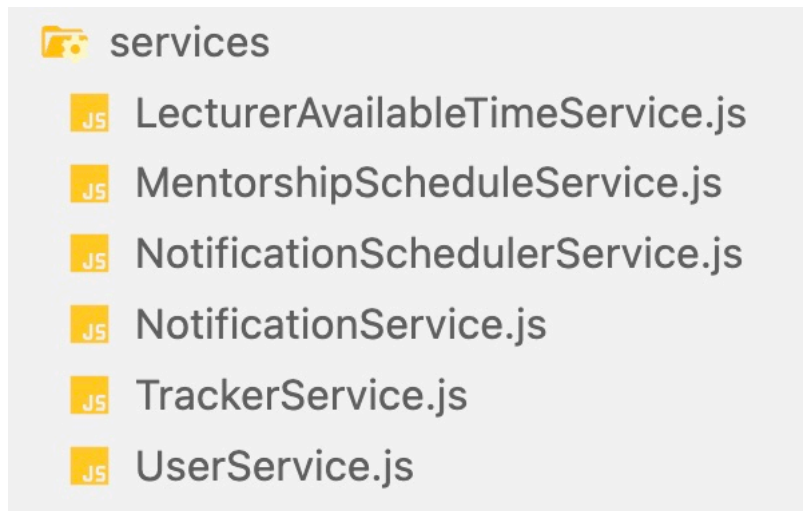
minimal pada bagian lainnya, sehingga meningkatkan stabilitas dan mengurangi risiko kesalahan.

Dalam konteks pengembangan proyek ini, penulis menerapkan prinsip SoC untuk memisahkan logika bisnis dalam sebuah transaksi ke dalam modul bernama *Service* untuk mengakomodasi kebutuhan-kebutuhan yang dikembangkan lebih lanjut. Berdasarkan hasil pengembangan awal, arsitektur desain yang digunakan tidak melewati modul *Service* untuk mengelola transaksi, melainkan dari *Controller* langsung dengan Model yang terkoneksi dengan basis data. Modul *Service* pada pengembangan lanjutan ini bertindak sebagai penghubung antara *Controller* yang mengelola permintaan pengguna dengan *Model* yang mengelola penyimpanan data. Secara sederhana, keterkaitan antara ketiga modul tersebut divisualisasikan pada diagram berikut.



Gambar 3.1 Hubungan Antara Modul dengan Prinsip *Separation of Concerns* (SoC)

Berdasarkan Gambar 3.1 di atas, terdapat modul *Service* yang merupakan pengembangan lanjutan dari sistem yang telah dikembangkan sebelumnya. Hal tersebut bertujuan untuk memastikan modularitas serta pemanfaatan kembali kode berulang yang memiliki logika bisnis serupa. Modul *Service* memuat implementasi aturan bisnis, validasi data yang lebih mendalam, dan orkestrasi interaksi dengan modul *Model* basis data. Modul tersebut menerima data dari *Controller*, memprosesnya sesuai dengan kebutuhan aplikasi, dan berinteraksi dengan Model untuk menyimpan atau mengambil informasi dari basis data. Dalam implementasinya, terdapat tujuh (7) *service* yang berhasil dikembangkan guna mengelola kebutuhan logika transaksi yang diperlukan.



Gambar 3.2 Modul *Service* yang Dikembangkan

Berdasarkan Gambar 3.2 di atas, tersedia tujuh (7) *service* seperti “LecturerAvailableTimeService.js” yang mengelola informasi ketersediaan waktu membimbing dosen pembimbing, “LogbookItemService.js” yang menangani pengelolaan catatan *logbook* mahasiswa, “NotificationSchedulerService.js” dan “NotificationService.js” yang mengelola sistem notifikasi terjadwal dan pengiriman notifikasi, “TrackerService.js” yang mengelola proses pencatatan ketika sesi bimbingan sedang berlangsung, dan “UserService.js” yang mengelola informasi terkait pengguna. Pemisahan antara *service* tersebut mencerminkan penerapan prinsip SoC dengan mengisolasi logika bisnis yang berbeda ke dalam modul-modul independen. Adapun pemisahan tersebut membuat logika bisnis yang sama dapat digunakan secara berulang pada proses transaksi yang berbeda dengan tetap mempertahankan algoritma yang terpisah pada setiap modulnya.

b. *Singleton*

Untuk memastikan hanya ada satu *instance* objek yang mengelola pengiriman dan penjadwalan notifikasi di keseluruhan lingkup aplikasi *back-end*, ‘NotificationSchedulerService’ diimplementasikan menggunakan pola desain *Singleton*. Pola

desain *Singleton* merupakan sebuah *creational pattern* (pola kreasi) yang digunakan untuk membatasi pembuatan objek dari sebuah kelas menjadi hanya satu objek tunggal (Rana dan Saleh 2022). Menurut Rana dan Saleh (2022), tujuan utama dari pola ini adalah untuk menyediakan satu titik akses global ke objek tersebut sehingga memastikan bahwa semua bagian dari sistem yang memerlukan fungsionalitasnya akan berinteraksi dengan objek yang sama. Dalam konteks proyek ini, pola *Singleton* menjamin bahwa hanya satu objek dari kelas 'NotificationScheduler' yang dibuat dan dapat diakses secara global. Dalam cuplikan kode, logika *Singleton* diterapkan di dalam konstruktor kelas "NotificationScheduler" seperti pada Gambar 3.3 berikut.

```
class NotificationScheduler {
  1 reference
  constructor() {
    if (!NotificationScheduler.instance) {
      /** @type {Array<Worker>} */
      this.workers = []
      this.config = { maxWorkers: 2 };
      this.nextWorkerIndex = 0;

      NotificationScheduler.instance = this;

      this.initializeWorkers()
    }
  }

  1 reference
  initializeWorkers() {
    for (let i = 0; i < this.config.maxWorkers; i++) {
      const worker = new Worker('./workers/notification-worker.js')

      this.workers.push(worker)
    }
  }

  2 references
  getWorker() {
    const worker = this.workers[this.nextWorkerIndex % this.workers.length];
    this.nextWorkerIndex++;

    return worker
  }
}
```

Gambar 3.3 Cuplikan Kode Implementasi Pola Desain Singleton

Ketika konstruktor dipanggil, maka akan dilakukan pemeriksaan apakah sudah ada *instance* yang tersimpan di "NotificationScheduler.instance". Jika tidak ada, *instance* akan dibuat, dan menginisialisasi variabel seperti array "workers",

konfigurasi “config”, dan indeks “nextWorkerIndex”. Dilanjutkan dengan penyimpanan *instance* yang berhasil dibuat di variabel bernama “NotificationScheduler.instance”. Namun, jika sudah ada *instance*, maka prosesnya adalah tidak akan membuat yang baru, melainkan mengembalikan yang sudah tersimpan sebelumnya. Hal ini terjadi untuk memastikan hanya satu objek yang pernah dibuat. Dengan demikian, semua modul lain pada aplikasi yang memerlukan pengiriman notifikasi akan berinteraksi dengan *instance* tunggal tersebut dengan harapan dapat mencegah konflik dan memastikan pengelolaan sumber daya yang konsisten, terutama dalam pengelolaan “Worker” pada *thread* terpisah.

c. *Worker Threads*

Implementasi *worker threads* dalam pengembangan proyek ini berperan penting dalam meningkatkan kinerja dan responsivitas aplikasi *back-end* yang berpotensi memblokir *thread* utama apabila mengerjakan pekerjaan yang berat dalam satu waktu. *Worker threads* memungkinkan eksekusi kode JavaScript secara paralel dengan memisahkan tugas dari *thread* utama yang mengolah data untuk ditampilkan pada *front-end*. Cuplikan kode dari bagaimana *worker threads* diimplementasi pada pengembangan proyek ini ditunjukkan pada Gambar 3.4 berikut.

```

parentPort.on('message', async ({ type, pushNotification }) => {
  try {
    if (type === 'DIRECT') {
      console.log("[WORKER] Running Direct Notification")
      await sendDirectWebPushNotification({
        userId: pushNotification.userId,
        title: pushNotification.title,
        body: pushNotification.body,
        data: pushNotification.data
      })
    } else if (type === NOTIFICATION_TYPE.REMINDER_MENTORING_30MIN) {
      console.log("[WORKER] Running Reminder Mentoring 30MIN Notification")
      await processReminderMentoringNotification()
    } else if (type === 'SCHEDULER') {
      console.log("[WORKER] Running Scheduler Notification")
      const [today, time] = getCurrentDayAndTimeLocale();
      const scheduledNotifications = await db.model(NotificationScheduler.name).findAll({
        attributes: ['id', 'day', 'time', 'repeat', 'ctaId', 'notificationTypeId', 'title', 'description'],
        where: {
          day: {
            [Op.or]: [today, null]
          },
          time,
        },
        include: ['notificationType'],
      })

      if (!scheduledNotifications.length > 0) {
        console.log("[WORKER] No scheduled notification at this time");
        return
      }

      if (scheduledNotifications.length > 0) {
        await Promise.allSettled(scheduledNotifications.map(
          (scheduler) => processScheduledNotification(scheduler)
        ))
      }
    }
  }
})

```

Gambar 3.4 Cuplikan Kode *Worker Thread*

Tangkapan layar yang berisikan cuplikan kode di atas menggunakan *worker threads* untuk menjalankan tugas terkait pengiriman notifikasi di latar belakang aplikasi, yang memungkinkan aplikasi utama untuk tetap responsif. Secara spesifik, *worker threads* ini dibuat untuk menangani pengiriman notifikasi *push* dan penjadwalan notifikasi. Komunikasi dengan *worker threads* dilakukan melalui “parentPort.on('message')”, yang berjalan ketika ada pesan dari *thread* utama. Ketika pesan diterima, *worker threads* menjalankan fungsi yang sesuai berdasarkan jenis notifikasi yang perlu diproses. Misalnya, jika pesan bertipe “DIRECT”, maka *worker threads* akan mengirimkan notifikasi *push* dan menambahkan notifikasi *in-app* langsung ke pengguna tertentu. Jika pesan bertipe “SCHEDULER”, maka *worker threads* akan mengambil notifikasi terjadwal dari basis data dan mengirimkannya pada waktu yang ditentukan. *Worker threads* juga menangani pembaruan basis data terkait notifikasi, seperti menyimpan detail

notifikasi dan menghapus kode identifikasi langganan notifikasi *push* yang tidak valid.

Untuk mendistribusikan tugas secara merata ke beberapa *worker threads*, digunakan algoritma *round robin*. Algoritma *round robin* adalah metode penjadwalan yang memberikan setiap proses jatah waktu eksekusi yang sama secara bergilir (Putra dan Purnomo 2022). Dalam konteks proyek ini, algoritma *round Robin* dimanfaatkan dengan memberikan distribusi tugas untuk setiap *worker* yang disimpan dalam sebuah *array* secara bergiliran untuk setiap tugasnya. Variabel “*nextWorkerIndex*” berguna untuk melacak *worker* mana yang akan menerima tugas selanjutnya. Ketika sebuah tugas perlu dikirim, fungsi “*getWorker()*” mengembalikan *worker* pada indeks “*nextWorkerIndex*” yang kemudian nilai dari “*nextWorkerIndex*” akan ditambah dengan modulus banyaknya jumlah *worker* dalam *array*.

```

class NotificationScheduler {
  1 reference
  constructor() {
    if (!NotificationScheduler.instance) {
      /** @type {Array<Worker>} */
      this.workers = [];
      this.config = { maxWorkers: 2 };
      this.nextWorkerIndex = 0;

      NotificationScheduler.instance = this;

      this.initializeWorkers()
    }
  }

  1 reference
  initializeWorkers() {
    for (let i = 0; i < this.config.maxWorkers; i++) {
      const worker = new Worker('./workers/notification-worker.js')
      this.workers.push(worker)
    }
  }

  2 references
  getWorker() {
    const worker = this.workers[this.nextWorkerIndex % this.workers.length];
    this.nextWorkerIndex++;
    return worker
  }

  dispatchDirectNotification(pushNotification) {
    const worker = this.getWorker();

    // Sent to worker
    worker.postMessage({ type: 'DIRECT', pushNotification });
  }

  1 reference
  triggerCronNotification() {
    const scheduledNotifications = [
      'SCHEDULER',
      'REMINDER_MENTORING_30MIN'
    ]

    while (scheduledNotifications.length > 0) {
      const worker = this.getWorker();
      const taskType = scheduledNotifications.shift();

      worker.postMessage({ type: taskType });
    }
  }

  0 references
  terminateWorkers() {
    this.workers.forEach(worker => {
      worker.terminate()
    })
  }
}

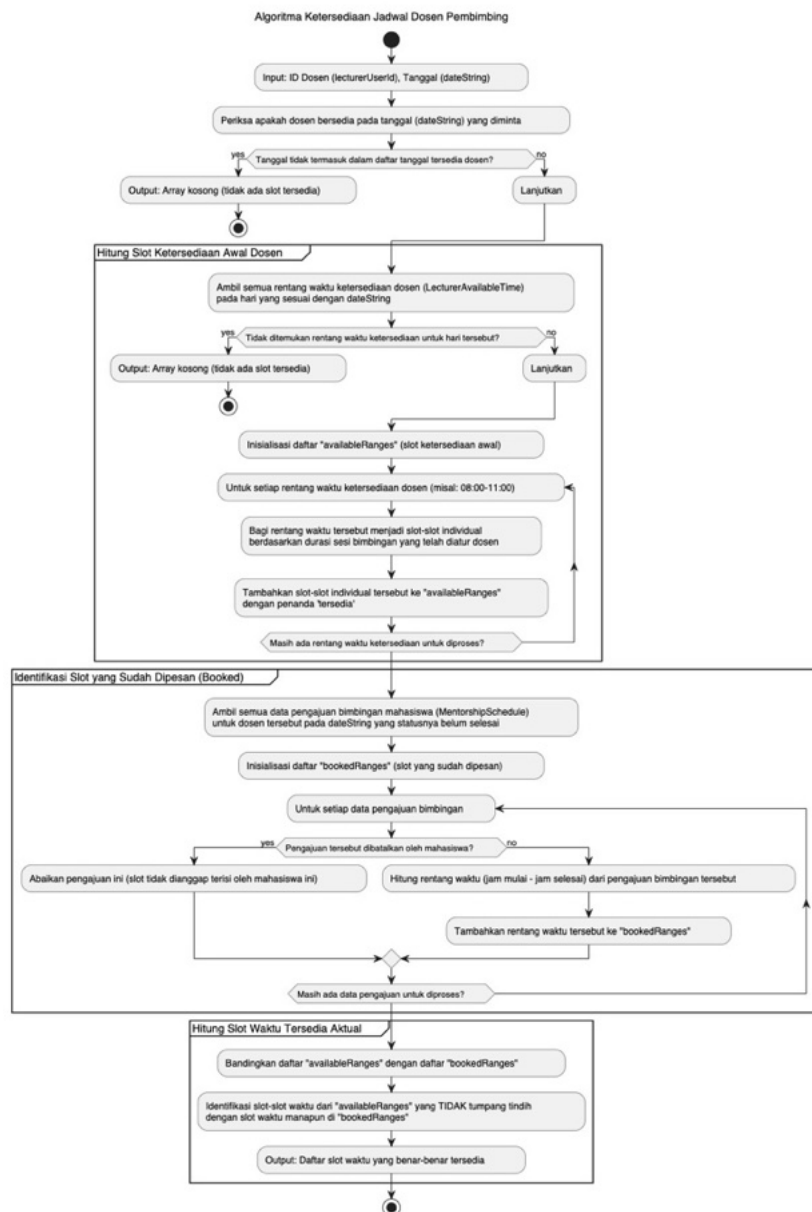
```

Gambar 3.5 Kode “NotificationSchedulerService” yang Mendistribusikan Tugas ke *Worker Thread*

Dengan cara ini, tugas-tugas didistribusikan secara bergantian ke setiap *worker*, memastikan tidak ada *worker* yang kelebihan beban. Alur kerja ini diinisialisasi oleh NotificationSchedulerService yang mengelola siklus hidup *worker threads*. Service ini membuat kumpulan *worker thread* saat dibangkitkan dan menggunakan metode “getWorker()” untuk menerapkan logika *round robin* ketika mendistribusikan pengiriman notifikasi. Dengan memindahkan tugas-tugas ke *worker threads*, aplikasi utama tidak terbebani dan dapat terus berjalan dengan lancar tanpa hambatan.

d. Algoritma Jadwal Ketersediaan Dosen Pembimbing

Algoritma ini bertugas untuk secara cermat menghitung dan menampilkan slot waktu dosen pembimbing yang tersedia pada tanggal tertentu. Proses ini melibatkan analisis terhadap jadwal ketersediaan yang telah diatur oleh dosen pembimbing, digabungkan dengan data bimbingan yang sudah terjadwalkan oleh mahasiswa, serta mempertimbangkan pembatalan pengajuan yang mungkin terjadi. Berikut merupakan visualisasi alur kerja mengenai algoritma penentuan ketersediaan jadwal dosen pembimbing yang telah dikembangkan.



Gambar 3.6 Visualisasi Algoritma Penentuan Jadwal Ketersediaan Dosen Pembimbing

Sesuai dengan visualisasi pada diagram Gambar 3.6 di atas, alur kerja algoritma penentuan ketersediaan jadwal diawali dengan penerimaan masukkan berupa identitas dosen dan tanggal spesifik yang ingin diperiksa. Langkah pertama adalah melakukan validasi apakah dosen tersebut memang memiliki ketersediaan jam membimbing pada tanggal yang diminta. Jika ada, maka sistem melanjutkan ke tahap berikutnya, yaitu menghitung seluruh slot ketersediaan awal dosen dengan

mengambil rentang waktu yang telah diatur oleh dosen untuk hari tersebut dan memecahnya menjadi slot individual sesuai durasi sesi bimbingan. Selanjutnya, algoritma akan mengidentifikasi semua slot waktu yang sudah terjadwal untuk bimbingan dengan mahasiswa lain pada tanggal yang sama, dengan juga memperhitungkan status pembatalan pengajuan oleh mahasiswa yang menganggap slot tersebut tidak terisi. Tahap akhir adalah proses kalkulasi irisan antara daftar slot ketersediaan awal dengan daftar slot yang sudah terjadwal untuk menghasilkan daftar final berisi slot waktu yang benar-benar tersedia dan dapat dipilih oleh mahasiswa untuk melakukan pengajuan bimbingan.

3.2.2. Implementasi *Front-end*

a. *Service Worker*

Dalam pengembangan proyek ini, *Service Worker* memegang peranan penting dalam meningkatkan fungsionalitas seperti notifikasi *push* dan kinerja aplikasi pada perangkat *mobile*. Implementasi yang dilakukan secara sederhana penerapan *service worker* pada proyek ini memaksimalkan fungsi untuk penerimaan dan menampilkan notifikasi *push*, serta penyimpanan *cache* guna meningkatkan kinerja aplikasi. Untuk penerapan penyimpanan *cache*, divisualisasikan pada Gambar 3.7 di bawah ini.

```

sw.addEventListener('install', (event) => {
  console.log('installing service worker for version', version);

  // Create a new cache and add all files to it
  1 reference
  async function addFilesToCache() {
    const cache = await caches.open(CACHE);

    console.log('Attempting to cache assets individually...');

    const results = await Promise.allSettled(
      ASSETS.map(async (asset) => {
        try {
          await cache.add(asset);
          console.log(`Cached: ${asset}`);
          return { status: 'fulfilled', value: undefined };
        } catch (error) {
          console.error(`Failed to cache ${asset}:`, error);
          return { status: 'rejected', reason: error };
        }
      })
    );

    const rejected = results.filter((result) => result.status === 'rejected');
    if (rejected.length > 0) {
      console.warn('Some assets failed to cache:', rejected);
    } else {
      console.log('All assets cached successfully.');
```

Gambar 3.7 Proses Penyimpanan Data untuk *Cache* Ketika *Service Worker* Pertama Kali Melakukan Instalasi

Adapun proses pengambilan data dari penyimpanan *Cache* ditampilkan pada Gambar 3.8 berikut.

```

sw.addEventListener('fetch', (event) => {
  // ignore POST requests etc
  if (
    event.request.method !== 'GET' ||
    !event.request.url.startsWith('http') ||
    event.request.url.startsWith('chrome-extension') ||
    event.request.url.includes('extension')
  ) {
    return;
  }

  // reference
  async function respond() {
    const url = new URL(event.request.url);
    const cache = await caches.open(CACHE);

    // 'build'/'files' can always be served from the cache
    if (ASSETS.includes(url.pathname)) {
      const response = await cache.match(url.pathname);

      if (response) {
        return response;
      }
    }

    // for everything else, try the network first, but
    // fall back to the cache if we're offline
    try {
      const response = await fetch(event.request);

      // if we're offline, fetch can return a value that is not a Response
      // instead of throwing - and we can't pass this non-Response to respondWith
      if (!(response instanceof Response)) {
        throw new Error('invalid response from fetch');
      }

      if (response.status === 200) {
        cache.put(event.request, response.clone());
      }

      return response;
    } catch (err) {
      const response = await cache.match(event.request);

      if (response) {
        console.log('Returning from Cache', event.request.url);
        return response;
      }

      // if there's no cache, then just error out
      // as there is nothing we can do to respond to this request
      throw err;
    }
  }

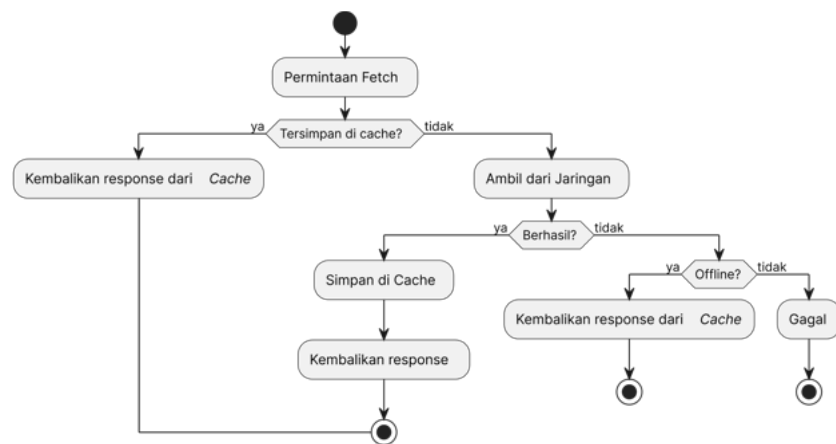
  event.respondWith(respond());
});

```

Gambar 3.8 Proses Pengambilan Data dari Penyimpanan *Cache*

Tangkapan layar kode program pada Gambar 3.7 dan Gambar 3.8 menggambarkan tiga bagian utama dari siklus hidup *service worker*, yaitu, *install*, *activate*, dan *fetch*. Pada saat *service worker* pertama kali melakukan instalasi, proses awal yang dilakukan adalah dengan membuat *cache* dan menyimpan aset-aset penting dari aplikasi. Proses ini sangat krusial untuk memastikan aplikasi dapat berfungsi secara *offline*. Fungsi “*addFilesToCache()*” secara asinkron menambahkan setiap aset dari daftar *ASSETS* ke dalam *cache* bernama *CACHE*. “*Promise.allSettled()*” juga digunakan untuk menangani setiap penyimpanan *cache* satu persatu, memastikan apabila terjadi kegagalan dalam menyimpan satu aset tidak menggagalkan seluruh proses instalasi. Setelah instalasi berhasil, *service worker* diaktifkan. Pada tahap ini, proses yang dilakukan adalah

membersihkan *cache* lama dari versi sebelumnya menggunakan fungsi “deleteOldCaches()”. Hal ini penting untuk memastikan bahwa aplikasi menggunakan versi aset-aset terbaru. Terakhir, adalah proses *fetch*. Setiap kali aplikasi melakukan permintaan HTTP untuk mengambil data seperti, HTML, CSS, atau gambar, *service worker* mencegat permintaan tersebut. Kode dalam event *fetch* menentukan bagaimana permintaan tersebut. Diagram mengenai proses yang telah dikembangkan divisualisasikan pada Gambar 3.9 di bawah.

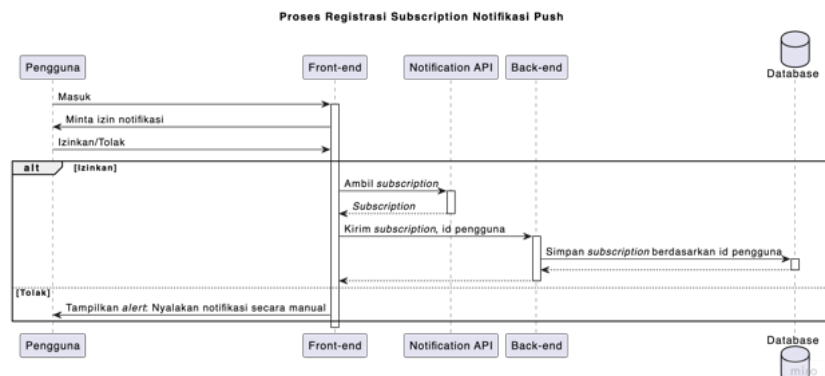


Gambar 3.9 Proses Instalasi, Aktivasi, dan *Fetch* pada *Service Worker*

Pertama, memeriksa apakah permintaan tersebut adalah untuk aset yang telah tersimpan di dalam *cache*. Jika tersimpan, maka *service worker* langsung mengembalikan data dari *cache*. Jika tidak tersimpan, maka *service worker* mencoba mengambil data secara langsung. Jika berhasil, sumber daya yang baru diambil ini akan disimpan juga ke dalam *cache* untuk penggunaan berikutnya. Jika pengambilan dari jaringan gagal, *service worker* mencoba mengambil data dari *cache*. Apabila data yang diminta tidak tersedia di dalam *cache*, maka permintaan tersebut gagal. Dengan demikian, aplikasi dapat memuat aset dengan cepat saat *online*, berfungsi secara *offline* dengan mengambil aset dari *cache*, dan selalu mendapatkan versi terbaru saat pengguna *online*.

b. Registrasi Notifikasi *Push*

Implementasi fitur notifikasi *push* pada proyek ini bertujuan untuk penyampaian informasi penting secara *real-time*. Proses registrasi notifikasi *push* memanfaatkan teknologi *Web Push*, sebuah standar yang memungkinkan aplikasi web mengirimkan notifikasi kepada pengguna meskipun aplikasi tidak sedang aktif atau terbuka di peramban. Mekanisme registrasi ini diawali dengan permintaan izin dari pengguna untuk menerima notifikasi *push*. Setelah izin diberikan, peramban akan menghasilkan sebuah *push subscription endpoint* unik untuk pengguna tersebut dan sistem *back-end*. Secara sederhana proses registrasi notifikasi *push* yang terjadi divisualisasikan dalam diagram berikut.



Gambar 3.10 Diagram Alur Registrasi Penerimaan Notifikasi *Push*

Diagram alur kerja pada Gambar 3.10 di atas menguraikan proses registrasi notifikasi *push* dari perspektif interaksi antar komponen sistem. Proses diawali dengan pengguna berinteraksi dengan *Front-end*, yang kemudian tampil konfirmasi izin kepada pengguna untuk mengaktifkan notifikasi *push*. Berdasarkan respons pengguna, terdapat kemungkinan dua alur yang mungkin terjadi. Jika pengguna memberikan izin, maka *Front-end* akan berinteraksi dengan *Notification API* milik peramban untuk mendapatkan objek *subscription* unik untuk perangkat dan pengguna tersebut. Objek *subscription* ini

kemudian dikirimkan oleh *Front-end* ke *Back-end*. Ketika *Back-end* menerima informasi *subscription* pengguna, maka informasi tersebut akan disimpan ke dalam *Database* dan mengasosiasikannya dengan ID pengguna terkait. Dengan demikian, *Back-end* memiliki informasi yang dibutuhkan untuk mengirimkan notifikasi *push* ke pengguna. Sebaliknya, jika pengguna menolak permintaan izin notifikasi *push*, *Front-end* akan menampilkan pemberitahuan kepada pengguna yang menginformasikan bahwa mereka perlu mengaktifkan notifikasi secara manual melalui pengaturan browser jika ingin menerima pembaruan secara *real-time*.

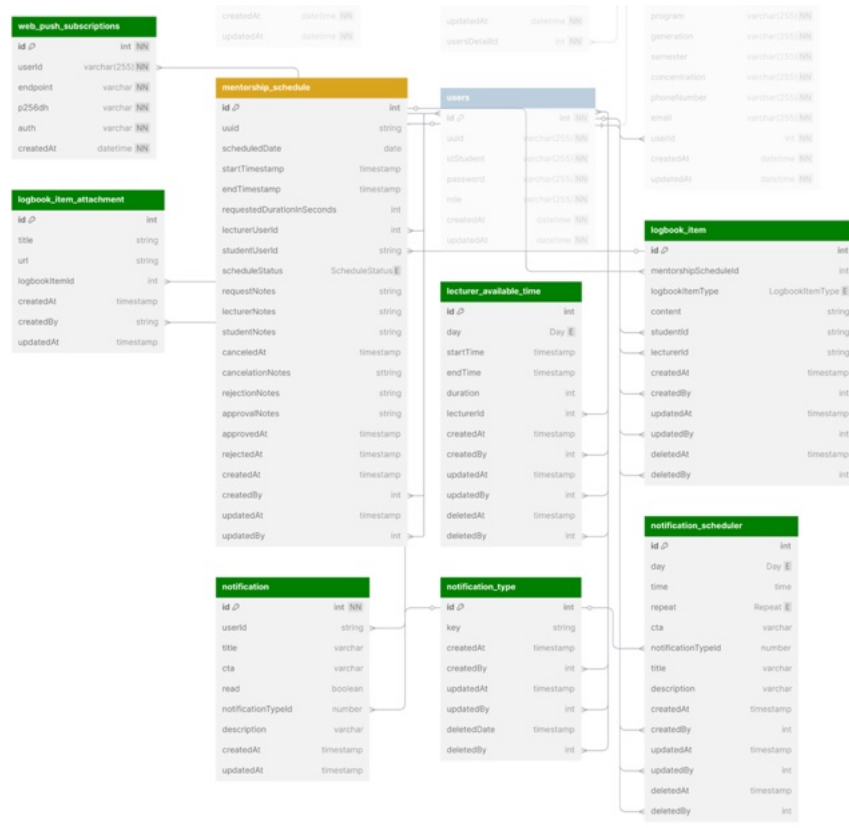
3.3. Metadata

Bagian ini menjelaskan informasi terkait data-data lain atau informasi tambahan yang berkaitan dengan pengembangan sistem yang telah dilakukan.

3.3.1. Basis Data

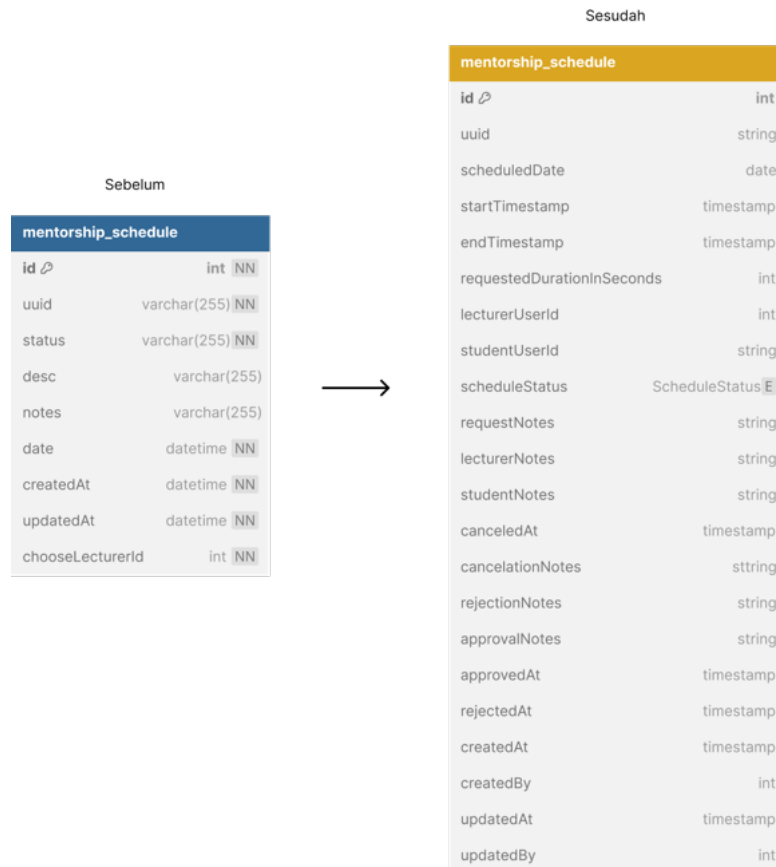
Penyesuaian dan pengembangan struktur basis data dilakukan untuk mengoptimalkan fungsionalitas PWA ini. Perubahan ini meliputi modifikasi pada tabel yang sudah ada untuk mengakomodasi fitur-fitur baru PWA, serta penambahan tabel baru untuk mendukung fungsionalitas tambahan. Tujuannya adalah untuk memastikan sistem dapat menyimpan dan mengelola data secara efektif, sehingga meningkatkan kinerja dan pengalaman pengguna secara keseluruhan. Terdapat tujuh (7) buah tabel baru yang ditambahkan untuk mendukung fungsionalitas tambahan meliputi, “web_push_subscription”, untuk menyimpan kode identifikasi pengiriman notifikasi *push* ke perangkat pengguna; “notification”, untuk menyimpan pemberitahuan terkini; “notification_type”, merupakan tabel turunan yang menyimpan jenis notifikasi; “notification_scheduler”, merupakan tabel untuk menyimpan notifikasi terjadwal; “logbook_item”, untuk menyimpan data-data

logbook mahasiswa; dan “lecturer_available_time”, untuk menyimpan data ketersediaan jadwal membimbing dosen. Adapun perubahan yang dilakukan pada tabel “mentorship_schedule” dengan pembaruan supaya dapat menyimpan data yang lebih komprehensif. Detail mengenai penambahan tabel diuraikan pada diagram Gambar 3.11 dengan penanda entitas berwarna hijau.



Gambar 3.11 Diagram Tujuh (7) Tabel Yang Ditambahkan

Adapun perubahan kolom pada tabel “mentorship_schedule” digambarkan pada Gambar 3.12 di bawah yang memperlihatkan penambahan kolom dengan tujuan agar data yang tersimpan lebih komprehensif dan informatif.



Gambar 3.12 Diagram Perubahan Tabel "mentorship_schedule"

Berdasarkan Gambar 3.12 di atas, terlihat perubahan yang cukup signifikan guna mengakomodasi kebutuhan yang lebih detail dan terstruktur. Tabel awal di sebelah kiri tampak lebih sederhana dengan fokus pada identifikasi unik (id, uuid), deskripsi (desc), catatan (notes), status (status), tanggal (date), dan informasi waktu pembuatan serta pembaruan (createdAt, updatedAt), serta identifikasi dosen yang dipilih (chooseLecturerId). Sementara tabel yang diperbarui sebelah kanan menunjukkan penambahan kolom-kolom yang lebih spesifik, meliputi pemisahan informasi waktu menjadi "scheduledDate", "startTimestamp", dan "endTimestamp", sehingga memungkinkan penjadwalan dan pelacakan sesi bimbingan dengan ketepatan waktu yang lebih baik. Selain itu, terdapat penambahan "requestedDurationInSeconds", yang mengindikasikan adanya pertimbangan durasi bimbingan yang diajukan oleh mahasiswa.

Tentu, mari kita telaah perubahan pada tabel *mentorship_schedule* yang Anda berikan.

Identifikasi peserta bimbingan juga diperjelas dengan adanya kolom “*lecturerUserId*” dan “*studentUserId*”. Penambahan kolom-kolom seperti “*requestNotes*”, “*lecturerNotes*”, “*studentNotes*”, “*canceledAt*”, “*cancellationNotes*”, “*rejectionNotes*”, “*approvedNotes*”, “*approvedAt*”, dan “*rejectedAt*” menunjukkan mekanisme pencatatan yang lebih komprehensif terkait proses pengajuan, persetujuan, penolakan, dan pembatalan bimbingan, termasuk alasan atau catatan terkait. Terakhir, informasi mengenai siapa yang membuat dan memperbarui data (*createdBy*, *updatedBy*) juga ditambahkan, meningkatkan aspek audit dan pelacakan perubahan dalam sistem.

3.3.2. Deskripsi Jenis Notifikasi

Terkait dengan proses pengiriman notifikasi, terdapat beberapa jenis notifikasi awal sebagai pendukung fitur yang dikembangkan. Terdapat dua jenis notifikasi, yaitu notifikasi yang dikirim ketika pengguna melakukan sebuah interaksi dan notifikasi terjadwal yang dikirim secara berkala. Kedua jenis notifikasi tersebut diuraikan pada kedua tabel di bawah ini.

Tabel 3.1 Daftar Jenis Notifikasi Berdasarkan Interaksi Pengguna

No.	Jenis Notifikasi	Jenis Interaksi	Target Pengiriman
1.	NEW_UPD ATE	Ketika pengguna mahasiswa menambahkan <i>update</i> terbaru di dalam <i>logbook</i> miliknya	Dosen Pembimbing 1 dan Dosen Pembimbing 2
2.	NEW_CO MMENT	Ketika pengguna dosen pembimbing menambahkan komentar pada <i>logbook</i> mahasiswa	Mahasiswa Terkait
3.	APPOINT MENT_RE QUESTED	Ketika mahasiswa mengajukan jadwal bimbingan	Dosen Pembimbing Terkait

No.	Jenis Notifikasi	Jenis Interaksi	Target Pengiriman
4.	APPOINTMENT_APPROVAL	Ketika dosen pembimbing menyetujui pengajuan jadwal bimbingan mahasiswa	Mahasiswa Terkait
5.	APPOINTMENT_REJECTED	Ketika dosen pembimbing menolak pengajuan jadwal bimbingan mahasiswa	Mahasiswa terkait
6.	APPOINTMENT_CANCELLED	Ketika dosen pembimbing atau mahasiswa membatalkan pengajuan jadwal bimbingan	Mahasiswa atau dosen pembimbing terkait

Tabel 3.2 Daftar Jenis Notifikasi Terjadwal

No.	Jenis Notifikasi	Waktu Pengiriman	Deskripsi Notifikasi Terjadwal	Target Pengiriman
1.	REMINDE R_UPDATE	Setiap hari Jumat pukul 17:00 WIB	Pengingat untuk mahasiswa menambahkan <i>update</i> terkait pengerjaan tugas akhir miliknya di dalam <i>logbook</i>	Mahasiswa
2.	REMINDE R_MENTORING_30MIN	30 menit sebelum jadwal sesi bimbingan dimulai	Pengingat untuk mahasiswa dan dosen pembimbing terkait jadwal bimbingan yang telah disetujui	Mahasiswa dan dosen pembimbing
3.	SCHEDULER_MENTORING_SOW	Setiap hari Senin pukul 08:00 WIB	Pengingat untuk menjadwalkan bimbingan dengan dosen pembimbing setiap minggunya	Mahasiswa
4.	SCHEDULER_SETTING_AVAILABLE_TIME_SOW	Setiap hari Senin pukul 08:00 WIB	Pengingat untuk mengatur ketersediaan waktu membimbing mahasiswa setiap minggunya	Dosen Pembimbing

3.3.3. Web Manifest

Web Manifest merupakan sebuah deklarasi terstruktur yang memberikan informasi penting kepada peramban tentang bagaimana sebuah aplikasi web seharusnya "di-install" ke layar utama pengguna, bagaimana aplikasi terlihat dan berperilaku setelah berhasil di-install, serta deklarasi penting seperti nama, deskripsi, ikon, dan warna tema. *File* ini memungkinkan situs web untuk memiliki pengalaman pengguna yang lebih mirip dengan aplikasi *native* sehingga dapat menjembatani kesenjangan antara fungsionalitas web dan aplikasi *mobile*.

```
{
  "name": "Lentera FIK UPNVJ",
  "description": "Layanan Elektronik Tugas Akhir Terintegrasi Fakultas Ilmu Komputer Universitas Pembangunan Nasional Veteran Jakarta",
  "short_name": "Lentera FIK",
  "display": "standalone",
  "start_url": "/?display_mode=standalone",
  "background_color": "#f8f8f8",
  "theme_color": "#f8f8f8",
  "icons": [
    {
      "src": "/app-icon-192x192.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/app-icon-512x512.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

Gambar 3.13 File Web Manifest Pada Sistem yang Dikembangkan

Berdasarkan konfigurasi *Web Manifest* pada Gambar 3.13 di atas, *metadata* PWA untuk sistem yang dikembangkan secara eksplisit mendefinisikan identitas dan tampilan aplikasi. Properti "name" dan "short_name" menetapkan nama lengkap dan nama pendek aplikasi, yang digunakan dalam berbagai konteks seperti layar utama dan daftar aplikasi pada perangkat pengguna. Deskripsi yang diberikan ("Layanan Elektronik Tugas Akhir Terintegrasi Fakultas Ilmu Komputer Universitas Pembangunan Nasional Veteran Jakarta") memberikan konteks yang jelas mengenai fungsi aplikasi. Properti "display" diatur ke "standalone", mengindikasikan bahwa aplikasi akan dibuka dalam jendela terpisah yang menyerupai aplikasi *native*.

URL awal (*start_url*) mengarah ke `"/?display_mode=standalone"`, yang menjadi titik masuk aplikasi saat diluncurkan. Warna latar belakang (*background_color*) dan warna tema (*theme_color*) keduanya ditetapkan ke `"#f8f8f8"` untuk memberikan konsistensi warna aplikasi. Adapun daftar ikon yang mendefinisikan ikon aplikasi dalam dua ukuran (192x192 dan 512x512 piksel) untuk tampilan yang optimal di berbagai perangkat dan konteks instalasi aplikasi pada jenis perangkat pengguna.

3.4. Laporan Implementasi Proyek

Bagian ini menyajikan *Logbook* atau catatan implementasi proyek. *Logbook* ini bertujuan untuk melacak kemajuan, mencatat penyelesaian tugas-tugas teknis, serta mendokumentasikan keputusan penting yang diambil selama fase implementasi. Adapun proses implementasi proyek dibagi menjadi dua (2) sistem yang berbeda antara *back-end* yang mengelola proses penyimpanan dan penyajian data dan *front-end* yang menyajikan tampilan interaktif hasil pengolahan data dari *back-end*.

3.4.1. *Logbook* Implementasi Proyek

Pencatatan pada *Logbook* mencakup berbagai aktivitas, mulai dari modifikasi basis data, pembaruan logika pada *endpoint* API, hingga implementasi fitur pada antarmuka pengguna, yang secara menggambarkan perkembangan pengerjaan sistem lanjutan menuju versi final. Uraian mengenai aktivitas atau kegiatan yang dilakukan selama proses implementasi dijelaskan pada Tabel 3.3 berikut.

Tabel 3.3 *Logbook* Implementasi

No.	Tanggal	Kegiatan/Aktivitas	Hasil/Keluaran	Keterangan
1	6 September 2024	Undangan akses akun untuk kolaborasi melalui <i>repository</i> di situs GitHub dengan Staf Fakultas Ilmu Komputer UPNVJ	Akun penulis berhasil memiliki akses untuk melakukan kolaborasi termasuk memberikan perubahan terhadap kode program	-
2	7 September 2024 s.d 10 Desember 2025	Identifikasi kebutuhan fungsional dan nonfungsional dengan melakukan observasi dan mewawancarai beberapa pemangku kepentingan yang terkait dengan pengembangan sistem termasuk dosen yang menjadi dosen pembimbing dan mahasiswa, serta staf Fakultas Ilmu Komputer UPNVJ yang sebelumnya telah mengembangkan fundamental dari sistem ini	Pengetahuan terkait proses bimbingan tugas akhir secara konvensional yang sedang berjalan serta sistem digital yang sedang dikembangkan	-
3	24 Februari 2025	Pengembangan fitur <i>Progressive Web Apps</i> (PWA) pada sistem bagian <i>front-end</i>	Sistem berhasil mengimplemen tasi fitur <i>Progressive Web Apps</i> (PWA)	-

No.	Tanggal	Kegiatan/Aktivitas	Hasil/Keluaran	Keterangan
4	4 Maret 2025	Pengembangan fitur notifikasi <i>push</i> dan notifikasi <i>in-app</i>	Pada sisi <i>back-end</i> , logika bisnis dikembangkan untuk melakukan pengiriman notifikasi kepada perangkat pengguna secara langsung berdasarkan ketentuan tertentu. Sedangkan pada sisi <i>front-end</i> , dikembangkan tampilan antarmuka untuk menampilkan notifikasi <i>in-app</i> dan memproses kemunculan notifikasi <i>push</i> secara langsung pada perangkat pengguna	Migrasi basis data dilakukan untuk mendukung penyimpanan notifikasi pengguna, jenis notifikasi, dan jadwal notifikasi
5	19 Maret 2025	Pemfaktoran ulang kode program untuk mendukung tampilan <i>front-end</i> yang lebih responsif terhadap perangkat pengguna	Tampilan antarmuka mendukung lebih banyak ukuran perangkat termasuk tablet dan juga perangkat telepon genggam	-

No.	Tanggal	Kegiatan/Aktivitas	Hasil/Keluaran	Keterangan
6	8 April 2025	Pembaruan versi <i>framework</i> yang digunakan untuk mendukung kebutuhan komponen terkini	Versi <i>framework</i> yang lebih terkini	Hal ini dilakukan untuk mendukung penggunaan komponen terbaru dari <i>library</i> yang dipasang pada sistem
7	9 s.d. 14 April 2025	Pengembangan fitur penjadwalan bimbingan	Fitur penjadwalan bimbingan secara manual dan ketersediaan jadwal yang diatur oleh dosen pembimbing	Migrasi basis data dilakukan untuk menyimpan data ketersediaan jadwal membimbing dan penambahan kolom pada jadwal bimbingan yang diajukan oleh mahasiswa

No.	Tanggal	Kegiatan/Aktivitas	Hasil/Keluaran	Keterangan
8	15 s.d. 20 April 2025	Pengembangan fitur <i>Logbook</i> mahasiswa	Fitur <i>Logbook</i> mahasiswa yang memungkinkan pencatatan sesi bimbingan	Migrasi basis data dilakukan untuk menyimpan catatan bimbingan mahasiswa, catatan komentar dari pengguna dosen pembimbing, dan pembaruan dari pengguna mahasiswa
9	21 s.d. 25 April 2025	Pengembangan fitur pelacakan sesi bimbingan berlangsung	Fitur pelacakan sesi bimbingan berlangsung secara dua arah baik dari pengguna mahasiswa dan dosen pembimbing	Migrasi basis data dilakukan untuk menyimpan catatan sesi bimbingan berlangsung serta waktu mulai dan berakhirnya bimbingan

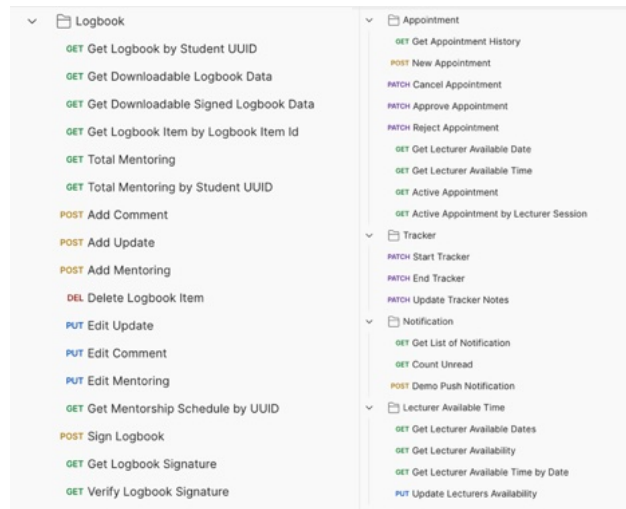
No.	Tanggal	Kegiatan/Aktivitas	Hasil/Keluaran	Keterangan
10	26 April s.d. 1 Mei 2025	Pengembangan fitur pengunduhan <i>Logbook</i> mahasiswa	Fitur pengunduhan <i>Logbook</i> mahasiswa	-
11	2 s.d. 13 Mei 2025	Perbaikan galat dan <i>bug</i> pada sistem	Sistem berjalan normal sesuai dengan kebutuhan yang telah teridentifikasi	-
12	14 Mei 2025	<i>User Acceptance Test</i> (UAT) terbatas terhadap salah satu pengguna dosen pembimbing	Pengujian berjalan dengan lancar, seluruh komponen pengujian berhasil dilaksanakan dan mendapat hasil sesuai yang diharapkan.	Terdapat masukan untuk pengembangan tanda tangan buku bimbingan secara digital untuk menghilangkan proses pencetakan kertas dan melakukan tanda tangan secara manual.

No.	Tanggal	Kegiatan/Aktivitas	Hasil/Keluaran	Keterangan
13	19 Mei 2025	<i>User Acceptance Test (UAT)</i> dengan pemangku kepentingan fakultas sebagai pengguna dosen pembimbing	Pengujian berjalan dengan lancar, seluruh komponen pengujian berhasil dilaksanakan dan mendapat hasil sesuai yang diharapkan.	Terdapat masukan seperti penyesuaian jumlah minimal bimbingan supaya lebih dinamis dan penambahan kop surat fakultas pada hasil pengunduhan buku bimbingan
14	20 s.d. 25 Mei 2025	<i>User Acceptance Test (UAT)</i> dengan mahasiswa sebagai pengguna mahasiswa	Pengujian berjalan dengan lancar, seluruh komponen pengujian berhasil dilaksanakan dan mendapat hasil sesuai yang diharapkan.	-

3.4.2. Hasil Implementasi *Back-end*

Implementasi pada sisi *back-end* mencakup penambahan *Application Programming Interface (API) endpoint* guna memudahkan proses konsumsi data oleh *front-end* serta mengakomodasi kebutuhan fungsionalitas tambahan dalam sistem

yang dikembangkan. Daftar mengenai API *endpoint* yang dibuat diuraikan pada Gambar 3.14 di bawah.



Gambar 3.14 Daftar API *Endpoint* Tambahan

Adapun proses migrasi basis data dilakukan guna mendukung penambahan fitur-fitur fungsional yang dikelola oleh seluruh API *endpoint* di atas. Selain perubahan dan penambahan, terdapat pula proses pengisian (*seeding*) data untuk beberapa tabel yang diperlukan. Perincian mengenai *file* migrasi yang dijalankan diuraikan pada tabel berikut.

Tabel 3.4 Daftar Migrasi Basis Data

No.	Nama <i>File</i>	Deskripsi Migrasi
1.	20250224134047-create-web-push-subscription.cjs	Penambahan tabel baru dengan nama “web_push_subscription” untuk menyimpan kode identifikasi perangkat pengguna yang menjadi target pengiriman notifikasi <i>push</i>
2.	20250317025126-create-lecturer-available-time.cjs	Penambahan tabel baru dengan nama “lecturer_available_time” untuk menyimpan data ketersediaan jadwal membimbing mahasiswa
3.	20250427061841-create-notification-type.cjs	Penambahan tabel baru dengan nama “notification_type” untuk menyimpan jenis-jenis notifikasi yang dikirim ke pengguna
4.	20250427065035-create-notification.cjs	Penambahan tabel baru dengan nama “notification” untuk menyimpan seluruh data notifikasi yang dikirimkan ke pengguna

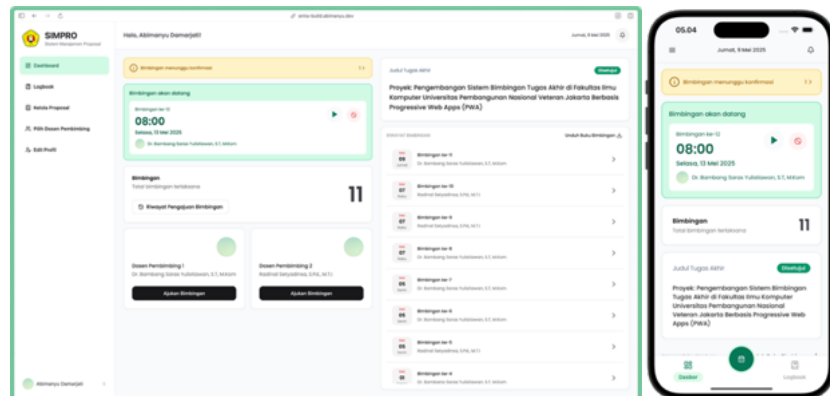
No.	Nama <i>File</i>	Deskripsi Migrasi
5.	20250427065816-create-notification-scheduler.cjs	Penambahan tabel baru dengan nama “notification_scheduler” untuk menyimpan notifikasi terjadwal
6.	20250427071818-create-logbook-item.cjs	Penambahan tabel baru dengan nama “logbook_item” untuk menyimpan isi konten dari <i>logbook</i> mahasiswa
7.	20250427071821-create-logbook-item-attachment.cjs	Penambahan tabel baru dengan nama “logbook_item_attachment” untuk menyimpan lampiran pada item <i>logbook update</i> atau komentar
8.	20250427075115-update-mentorship-schedule.cjs	Perubahan terkait nama kolom dan penambahan kolom baru pada tabel “mentorship_schedule”
9.	20250427062410-notification-type.cjs	Pengisian data awal untuk jenis-jenis notifikasi
10.	20250427070933-notification-scheduler.cjs	Pengisian data awal untuk notifikasi terjadwal

3.4.3. Hasil Implementasi *Front-end*

a. Dasbor Mahasiswa

Informasi pada laman ini hanya tersedia untuk pengguna mahasiswa yang telah disetujui topik proposalnya sesuai dengan visualisasi pada Gambar 3.15 di bawah. Pada laman tersebut terdapat informasi mengenai total bimbingan terlaksana, riwayat bimbingan, informasi mengenai tugas akhir, dan dosen pembimbing pilihan. Selain itu, terdapat bagian informasi dinamis yang menggambarkan jadwal bimbingan yang akan datang, yang sedang berlangsung, dan pengajuan yang sedang menunggu konfirmasi. Pengguna mahasiswa juga dapat mengajukan bimbingan langsung terhadap dosen pembimbing pilihannya pada versi *desktop*, sedangkan pada versi *mobile*,

pengguna dapat menekan tombol berwarna hijau yang ada di bawah layar.

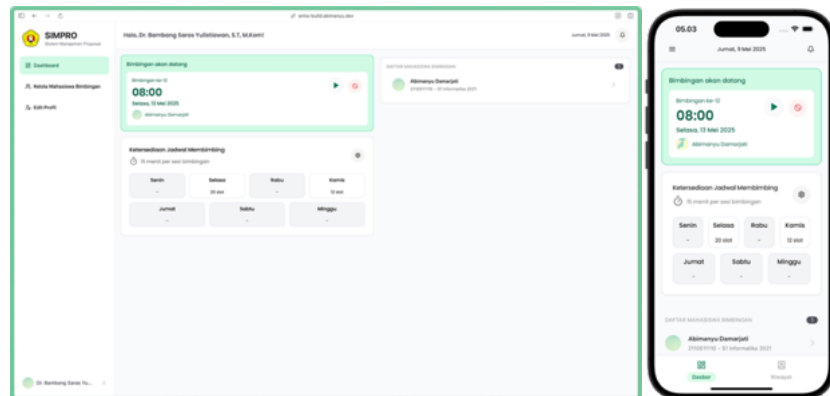


Gambar 3.15 Tampilan Dasbor Mahasiswa Versi *Desktop* (atas) dan Versi *Mobile* (bawah)

b. Dasbor Dosen Pembimbing

Informasi pada laman ini hanya tersedia untuk pengguna dosen pembimbing. Pada laman tersebut terdapat informasi mengenai status pengajuan bimbingan mahasiswa, daftar mahasiswa, dan ketersediaan jadwal membimbing. Pengguna dosen pembimbing juga dapat langsung mengatur ketersediaan jadwal membimbing melalui tombol pengaturan yang ada di dalam komponen tersebut. Selain itu, pengguna dosen pembimbing juga dapat memulai atau membatalkan bimbingan yang akan datang secara langsung melalui tombol coret berwarna merah, atau memulai sesi bimbingan dengan menekan tombol

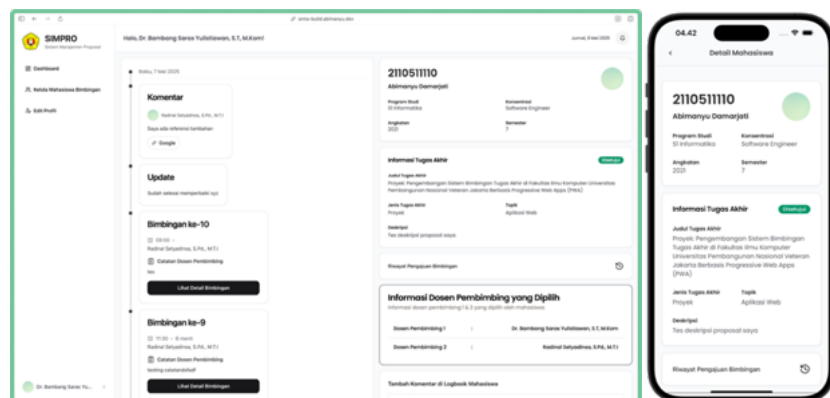
mulai berwarna hijau. Adapun tampilan terkait laman dasbor dosen pembimbing disajikan dalam Gambar 3.16 berikut.



Gambar 3.16 Tampilan Dasbor Dosen Pembimbing Versi *Desktop* (kiri) dan Versi *Mobile* (kanan)

c. Detail Mahasiswa

Laman ini merupakan laman khusus dosen pembimbing yang menyajikan informasi terkait kemajuan proses pengerjaan tugas akhir oleh mahasiswa tertentu. Pada laman ini, pengguna dosen pembimbing dapat melihat informasi mengenai detail mahasiswa, detail tugas akhir mahasiswa, *logbook* mahasiswa, dan informasi mengenai dosen pembimbing mahasiswa. Adapun pengguna dosen pembimbing dapat menambahkan komentar secara langsung terhadap kemajuan pengerjaan tugas akhir di dalam *logbook* mahasiswa. Adapun tampilan mengenai laman detail mahasiswa disajikan dalam Gambar 3.17 berikut.



Gambar 3.17 Tampilan Detail Mahasiswa Versi *Desktop* (kiri) dan Versi *Mobile* (kanan)

Abimanyu Damarjati, 2025

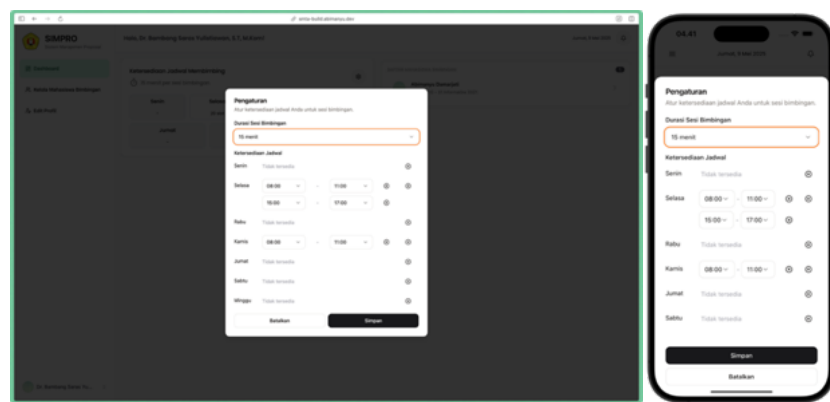
PROYEK: PENGEMBANGAN SISTEM BIMBINGAN TUGAS AKHIR DI FAKULTAS ILMU KOMPUTER UNIVERSITAS PEMBANGUNAN NASIONAL VETERAN JAKARTA BERBASIS PROGRESSIVE WEB APPS (PWA)

UPN Veteran Jakarta, Fakultas Ilmu Komputer, S1 Informatika

[www.upnvj.ac.id-www.library.upnvj.ac.id-www.repository.upnvj.ac.id]

d. Pengaturan Ketersediaan Jadwal Membimbing

Dialog yang tampil ketika pengguna dosen pembimbing menekan tombol pengaturan ketersediaan jadwal membimbing disajikan dalam Gambar 3.18 di bawah. Pada dialog tersebut, pengguna dosen pembimbing dapat mengatur durasi sesi bimbingan yang diinginkan serta pilihan jam yang tersedia pada hari-hari tertentu. Pengguna dapat menghapus juga ketersediaan jadwal membimbing sebelumnya atau menambahkan baru pada hari lainnya.

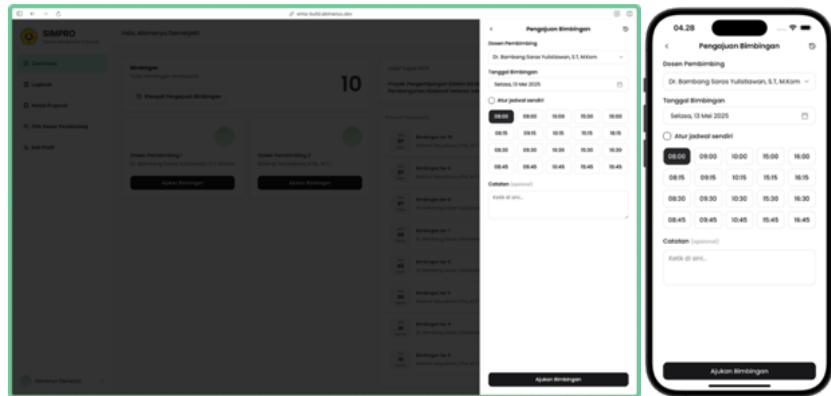


Gambar 3.18 Tampilan Pengaturan Ketersediaan Jadwal Membimbing Versi *Desktop* (kiri) dan Versi *Mobile* (kanan)

e. Pengajuan Jadwal Bimbingan

Bagian ini merupakan fitur pengguna mahasiswa yang memungkinkan untuk mengajukan jadwal bimbingan kepada dosen pembimbing pilihannya. Pengguna dapat memilih dosen pembimbing yang ingin diajukan sesi bimbingan, yang kemudian akan memuat data ketersediaan jadwal dosen pembimbing yang dipilih sesuai dengan ketersediaan jadwal membimbing yang sudah diatur oleh dosen pembimbing pilihan. Mahasiswa juga dapat menambahkan catatan pengajuan untuk disampaikan terhadap dosen bimbingan pilihannya. Adapun fitur lain yang memungkinkan mahasiswa mengajukan sesi bimbingan secara manual selain dari ketersediaan jadwal membimbing dosen ketika ada alasan tertentu. Adapun tampilan pengajuan jadwal

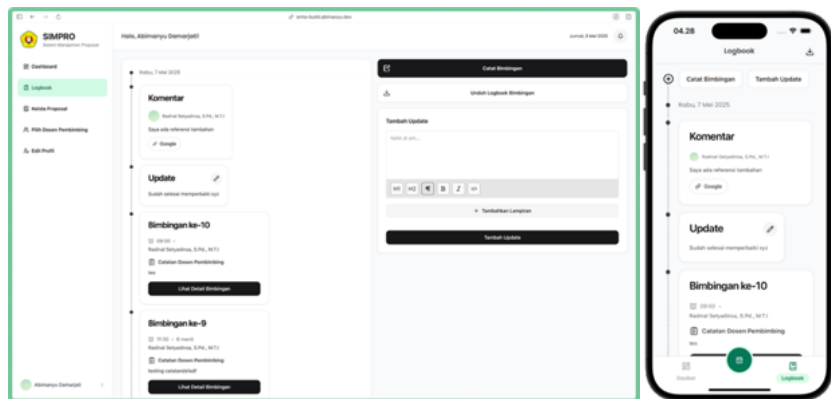
bimbingan oleh mahasiswa diperlihatkan dalam Gambar 3.19 berikut.



Gambar 3.19 Tampilan Pengajuan Jadwal Bimbingan oleh Mahasiswa Versi *Desktop* (kiri) dan Versi *Mobile* (kanan)

f. *Logbook* Mahasiswa

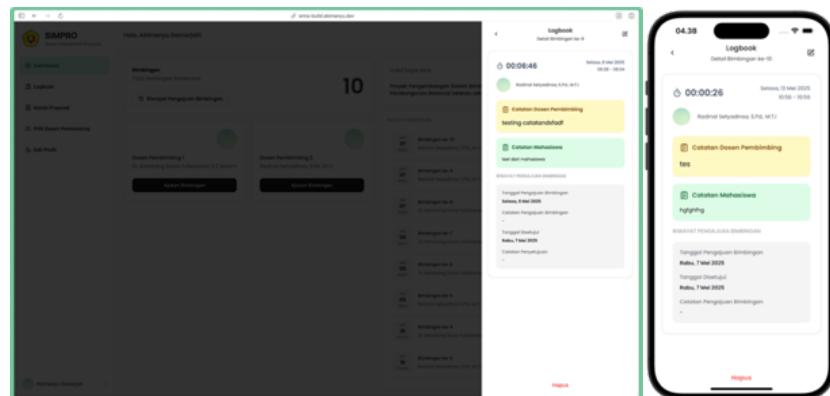
Laman ini merupakan fitur pengguna mahasiswa yang memungkinkan pengguna untuk melihat riwayat pengerjaan tugas akhir termasuk riwayat bimbingan, riwayat pembaruan, dan riwayat umpan balik dari dosen pembimbing. Pengguna dapat secara langsung menambahkan *update* dan mencatat bimbingan secara manual sebagai bentuk laporan kemajuan pengerjaan tugas akhir. Selain itu, pengguna juga dapat mengunduh *logbook* bimbingan untuk nantinya dicetak dan diajukan sebagai salah satu syarat untuk sidang proposal maupun sidang akhir. Tampilan terkait laman ini disajikan dalam Gambar 3.20 berikut.



Gambar 3.20 Tampilan *Logbook* Mahasiswa Versi *Desktop* (kiri) dan Versi *Mobile* (kanan)

g. Detail Catatan Bimbingan

Bagian ini berdasarkan Gambar 3.21 di bawah menyajikan informasi mengenai detail bimbingan yang telah terlaksana dan dapat diakses dua arah baik dari pengguna mahasiswa maupun pengguna dosen pembimbing. Informasi yang disajikan pada bagian ini mencakup, tanggal dan waktu sesi bimbingan dilakukan, durasi sesi bimbingan, informasi terkait dosen pembimbing yang melakukan pembimbingan, catatan baik dari mahasiswa ataupun dosen pembimbing, serta informasi terkait perubahan status pengajuan bimbingannya. Adapun mahasiswa dapat mengubah riwayat tersebut untuk disesuaikan kembali. Pengguna mahasiswa juga dapat menghapus riwayat tersebut apabila terdapat kesalahan dalam pengisian catatan bimbingan secara manual.

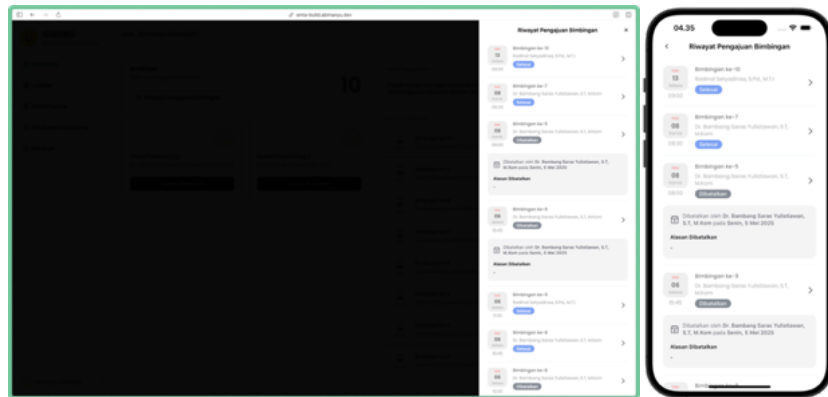


Gambar 3.21 Tampilan Detail Catatan Bimbingan Versi *Desktop* (kiri) dan Versi *Mobile* (kanan)

h. Riwayat Pengajuan Bimbingan

Bagian ini yang terlihat pada Gambar 3.22 di bawah menyajikan daftar informasi mengenai pengajuan bimbingan yang dilakukan oleh mahasiswa. Daftar riwayat bimbingan dapat diakses baik dari dosen pembimbing maupun mahasiswa untuk memudahkan pencarian informasi terkait proses bimbingan yang sebelumnya telah dilakukan. Adapun informasi yang disajikan pada bagian ini meliputi tanggal dan waktu bimbingan, siapa

dosen yang membimbing dari sisi pengguna mahasiswa atau siapa mahasiswa yang melakukan bimbingan dari sisi dosen pembimbing, serta informasi mengenai bimbingan ke-sekian dan status pengajuannya. Informasi lain mengenai alasan pembatalan atau alasan penolakan juga ditampilkan jika memang statusnya ditolak ataupun dibatalkan.

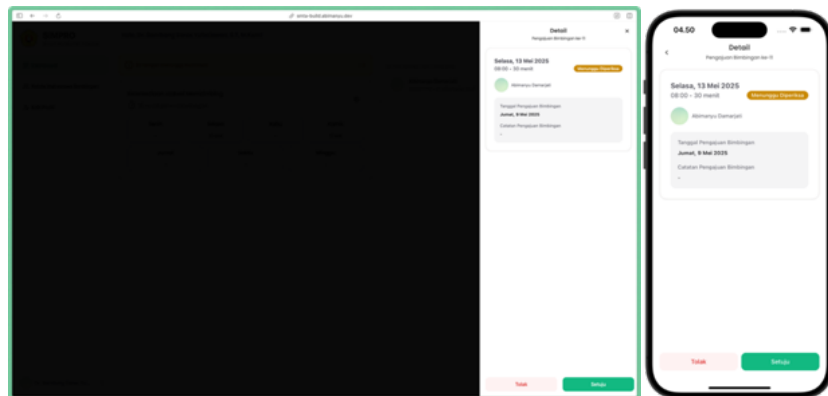


Gambar 3.22 Tampilan Riwayat Pengajuan Bimbingan Versi *Desktop* (kiri) dan Versi *Mobile* (kanan)

i. Detail Pengajuan Bimbingan

Bagian ini menyajikan informasi mengenai detail pengajuan bimbingan mahasiswa. Informasi yang disajikan sama dengan pada daftar riwayat pengajuan bimbingan seperti, tanggal, waktu, dan durasi bimbingan; informasi mengenai siapa yang mengajukan atau dosen pembimbing mana yang diajukan; dan catatan lain serta tanggal perubahan status pengajuan bimbingan. Adapun interaksi yang dapat dilakukan oleh pengguna dosen pembimbing adalah menolak, menyetujui pengajuan bimbingan, atau membatalkan pengajuan bimbingan yang sebelumnya telah disetujui. Sementara pengguna mahasiswa dapat membatalkan

pengajuan bimbingan yang telah diajukan. Tampilan mengenai komponen ini ditampilkan dalam Gambar 3.23 berikut.



Gambar 3.23 Tampilan Detail Pengajuan Bimbingan Versi *Desktop* (kiri) dan Versi *Mobile* (kanan)

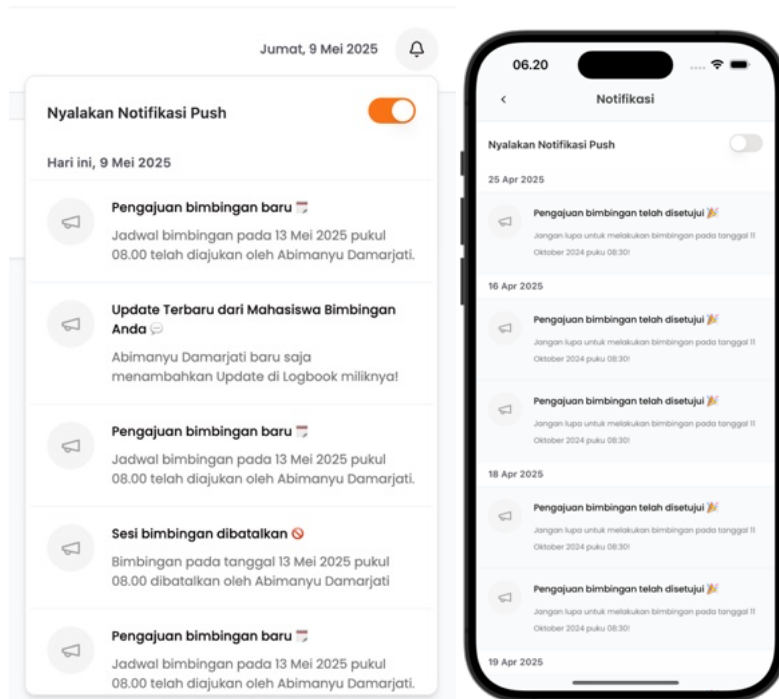
j. Notifikasi

Gambar 3.24 di bawah menampilkan notifikasi *push* yang diterima oleh perangkat pengguna baik dari *desktop* maupun *mobile*. Secara struktur, notifikasi *push* yang diterima dan ditampilkan oleh perangkat memiliki tiga (3) komponen seperti judul, deskripsi, dan ikon. Selain itu notifikasi *push* apabila ditekan juga akan mengarahkan pengguna ke halaman terkait.



Gambar 3.24 Tampilan Notifikasi Versi *Desktop* (kiri) dan Versi *Mobile* (kanan)

Adapun notifikasi *in-app* versi *desktop* dan *mobile* juga menampilkan konten judul dan deskripsi yang sama dengan notifikasi *push*. Selain itu, pengguna dapat menonaktifkan atau mengaktifkan fitur notifikasi *push* jika diperlukan. Tampilan mengenai komponen notifikasi *in-app* disajikan dalam Gambar 3.25 berikut.

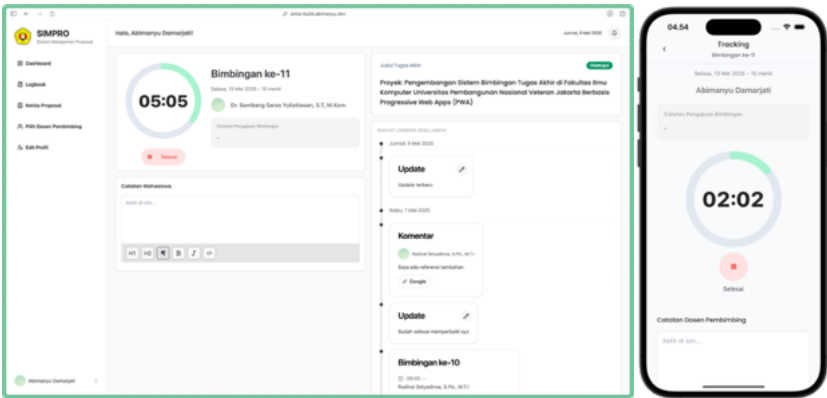


Gambar 3.25 Tampilan Komponen Notifikasi Versi *Desktop* (kiri) dan Versi *Mobile* (kanan)

k. Proses Bimbingan

Laman ini dapat diakses baik oleh mahasiswa maupun dosen pembimbing saat sesi bimbingan sedang dilaksanakan. Informasi yang disajikan meliputi riwayat *Logbook* sebelumnya guna memudahkan pemantauan proses pengerjaan, judul tugas akhir mahasiswa (untuk pengguna dosen pembimbing), dan waktu bimbingan berjalan. Pengguna juga dapat secara langsung mencatat proses bimbingan yang dilakukan dengan fitur simpan otomatis setiap terdapat perubahan pada kolom catatannya.

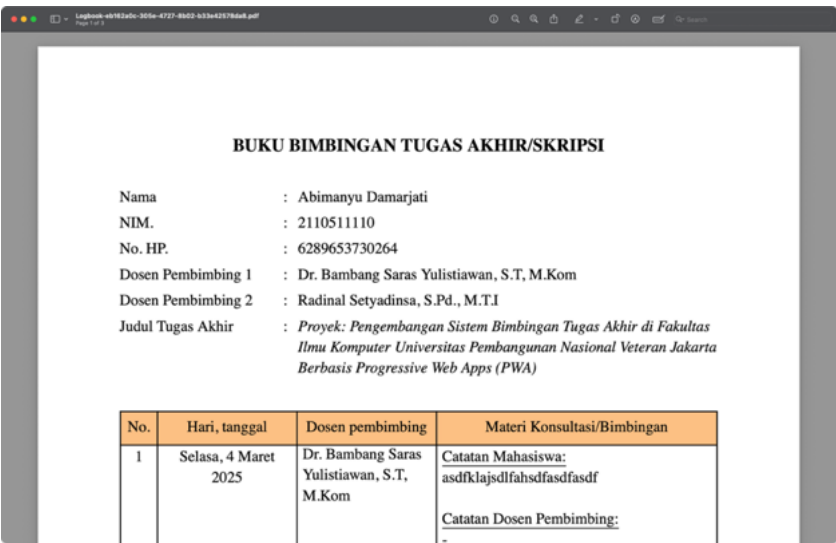
Tampilan proses bimbingan versi *desktop* dan *mobile* dapat disajikan dalam Gambar 3.26 berikut.



Gambar 3.26 Tampilan Proses Bimbingan Versi *Desktop* (kiri) dan Versi *Mobile* (kanan)

1. Hasil Pengunduhan Buku Bimbingan

Hasil unduhan buku bimbingan mahasiswa, yang ditampilkan pada Gambar 3.27 di bawah memuat detail informasi terkait mahasiswa dan tugas akhir miliknya, dilengkapi dengan catatan riwayat sesi bimbingan yang telah dilaksanakan. Penyusunan dokumen ini mengikuti format yang ditetapkan dalam Pedoman Tugas Akhir FIK Tahun 2024.... Selanjutnya, dokumen unduhan buku bimbingan tersebut dapat digunakan oleh mahasiswa sebagai salah satu kelengkapan persyaratan untuk melakukan pendaftaran seminar proposal maupun sidang akhir.



Gambar 3.27 Hasil Unduhan Buku Bimbingan Mahasiswa

3.5. Hasil Pengujian Proyek

Pengujian dilakukan menggunakan tiga (3) metode berbeda guna menguji kebutuhan fungsional dan nonfungsional. Uraian mengenai ketiga metode uji yang digunakan dijelaskan sebagai berikut.

3.5.1. *Black Box Testing*

Pengujian *black box* telah dilakukan pada saat implementasi dan integrasi dengan sistem sebelumnya yang telah dikembangkan dengan fokus pada fungsionalitas proses bimbingan tugas akhir. Terdapat 29 komponen pengujian yang dirancang untuk melakukan verifikasi berdasarkan masukan, keluaran, dan perilaku sistem sesuai dengan spesifikasi yang telah ditentukan di awal. Hasil pengujian menunjukkan bahwa sebagian besar fungsionalitas berjalan sesuai dengan kebutuhan, seperti yang terlihat pada pengujian modul notifikasi yang berhasil memproses pengiriman sejumlah notifikasi baik kepada mahasiswa dan dosen pembimbing dan menghasilkan pemberitahuan yang valid. Detail lengkap mengenai kasus uji yang dijalankan, hasil yang diharapkan, hasil aktual, dan status pengujian apakah berhasil atau gagal dapat dilihat pada Tabel 3.5 berikut.

Tabel 3.5 Hasil *Black Box Testing*

Kode	Deskripsi Pengujian	Skenario Pengujian	Input	Hasil yang Diharapkan	Skor Keberhasilan Target Pengguna	Keterangan
D1	Dosen pembimbing dapat mengatur ketersediaan waktu membimbing dalam satu minggu	1. Pengguna (dosen pembimbing) menekan tombol Pengaturan pada komponen “Ketersediaan Jadwal Membimbing” 2. Pengguna mengisi kolom ketersediaan durasi membimbing dengan memilih salah satu pilihan 3. Pengguna menambahkan ketersediaan waktu membimbing di hari tertentu	Durasi Membimbing: 30 menit Ketersediaan Membimbing: Selasa (08:00 – 11:00)	Pengguna mendapat tampilan konfirmasi berhasil memperbarui jadwal ketersediaan dan data pada komponen “Ketersediaan Jadwal Membimbing” juga sesuai dengan yang disimpan	4/4	Berhasil

Kode	Deskripsi Pengujian	Skenario Pengujian	Input	Hasil yang Diharapkan	Skor Keberhasilan Target Pengguna	Keterangan
D2	Dosen pembimbing dapat menyetujui pengajuan jadwal oleh mahasiswa	1. Pengguna (dosen pembimbing) menekan komponen informasi “Bimbingan menunggu konfirmasi” 2. Pengguna menekan salah satu pengajuan bimbingan dari daftar pengajuan bimbingan oleh mahasiswa 3. Pengguna menekan tombol “Setuju”	Catatan Persetujuan: “Ditunggu”	Pengguna (dosen pembimbing) mendapatkan tampilan konfirmasi berhasil menyetujui dan mahasiswa yang bersangkutan dapat melihat catatan persetujuan sesuai dengan masukan pengguna	4/4	Berhasil

Kode	Deskripsi Pengujian	Skenario Pengujian	Input	Hasil yang Diharapkan	Skor Keberhasilan Target Pengguna	Keterangan
D2	Dosen pembimbing dapat menolak pengajuan bimbingan	1. Pengguna (dosen pembimbing) menekan komponen informasi “Bimbingan menunggu konfirmasi” 2. Pengguna menekan salah satu pengajuan bimbingan dari daftar pengajuan bimbingan oleh mahasiswa 3. Pengguna menekan tombol “Tolak”	Catatan penolakan: “Berhalangan karena ada rapat”	Pengguna (dosen pembimbing) mendapatkan tampilan konfirmasi berhasil menolak dan mahasiswa yang bersangkutan dapat melihat catatan penolakan sesuai dengan masukan pengguna	4/4	Berhasil
D2	Dosen pembimbing dapat membatalkan jadwal bimbingan yang telah disetujui	1. Pengguna (dosen pembimbing) menekan tombol dengan ikon pembatalan berwarna merah di komponen “Daftar Bimbingan yang Akan Datang”	Alasan Pembatalan: “Ada rapat penting mendadak”	Pengguna (dosen pembimbing) mendapatkan konfirmasi pembatalan berhasil dilakukan dan data bimbingan yang dibatalkan hilang dari daftar bimbingan yang akan datang	4/4	Berhasil

Kode	Deskripsi Pengujian	Skenario Pengujian	Input	Hasil yang Diharapkan	Skor Keberhasilan Target Pengguna	Keterangan
D3	Dosen pembimbing menerima notifikasi terjadwal untuk memperbaiki ketersediaan jadwal membimbing	1. Pengguna (dosen pembimbing) mengizinkan pengiriman notifikasi <i>push</i>	Notifikasi terjadwal setiap hari Senin pukul 07:00 WIB	Pengguna (dosen pembimbing) mendapatkan notifikasi dan notifikasi <i>push</i> untuk memperbaiki jadwal ketersediaan membimbing sesuai dengan jam notifikasi terjadwal	4/4	Berhasil
D3	Dosen pembimbing menerima pembaruan <i>logbook</i> mahasiswa bimbingan	1. Pengguna (mahasiswa) menambahkan <i>update</i> pada <i>logbook</i> miliknya	Konten <i>Update</i> : "Menyesuaikan Bab II"	Dosen pembimbing menerima notifikasi <i>in-app</i> dan notifikasi <i>push</i> berisi informasi penambahan <i>update</i> terbaru dari mahasiswa	4/4	Berhasil

Kode	Deskripsi Pengujian	Skenario Pengujian	Input	Hasil yang Diharapkan	Skor Keberhasilan Target Pengguna	Keterangan
D3	Dosen pembimbing menerima notifikasi pengajuan bimbingan oleh mahasiswa	1. Pengguna (mahasiswa) mengajukan jadwal bimbingan	Pengajuan sesi bimbingan pada tanggal 8 Mei pukul 10:00	Dosen pembimbing menerima notifikasi <i>in-app</i> dan notifikasi <i>push</i> terkait pengajuan jadwal bimbingan sesuai masukan dari mahasiswa	4/4	Berhasil
D3	Dosen pembimbing menerima notifikasi perubahan status pengajuan bimbingan	1. Pengguna (mahasiswa) membatalkan pengajuan bimbingan	Alasan pembatalan: "Sakit"	Dosen pembimbing menerima notifikasi <i>in-app</i> dan notifikasi <i>push</i> terkait pembatalan pengajuan bimbingan	4/4	Berhasil
D3	Dosen pembimbing menerima notifikasi pengingat jadwal bimbingan yang akan datang	1. Pengguna (dosen pembimbing) mengizinkan pengiriman notifikasi <i>push</i>	Notifikasi terjadwal setiap 30 menit sebelum sesi bimbingan yang telah disetujui	Dosen pembimbing menerima notifikasi <i>in-app</i> dan notifikasi <i>push</i> terkait bimbingan yang akan datang tepat 30 menit sebelum waktu yang telah disetujui	4/4	Berhasil

D4	Dosen pembimbing memberikan komentar di dalam <i>logbook</i> milik mahasiswa bimbingan nya	<i>Desktop</i> 1. Pengguna (dosen pembimbing) menekan salah satu mahasiswa dari daftar mahasiswa bimbingan 2. Pengguna mengisi kolom komentar dan menambahkan lampiran pada <i>logbook</i> mahasiswa bimbingannya. <i>Mobile</i> 1. Pengguna (dosen pembimbing) menekan salah satu mahasiswa dari daftar mahasiswa bimbingan 2. Pengguna menekan tombol “Tambah Komentar” 3. Pengguna mengisi kolom komentar dan menambahkan lampiran pada <i>logbook</i> mahasiswa bimbingannya.	Komentar: “Bisa melakukan referensi ke website FIK” Lampiran: https://new-fik.upn-vj.ac.id	Pengguna (dosen pembimbing) mendapatkan konfirmasi berhasil menambahkan komentar ke dalam <i>logbook</i> mahasiswa dan dapat melihat komponen komentar tampil sesuai dengan masukan	4/4	Berhasil
----	--	--	---	---	-----	----------

Kode	Deskripsi Pengujian	Skenario Pengujian	Input	Hasil yang Diharapkan	Skor Keberhasilan Target Pengguna	Keterangan
D4	Dosen pembimbing dapat menghapus komentar miliknya dari <i>logbook</i> mahasiswa bimbingan	1. Pengguna (dosen pembimbing) menekan salah satu mahasiswa dari daftar mahasiswa bimbingan 2. Pengguna menekan tombol edit dengan ikon pensil di salah satu komentar miliknya 3. Pengguna menekan tombol hapus 4. Pengguna melakukan konfirmasi hapus komentar	-	Pengguna (dosen pembimbing) mendapatkan konfirmasi berhasil menghapus komentar dari <i>logbook</i> mahasiswa dan dapat melihat komponen komentar yang dihapus sudah tidak ada di dalam <i>logbook</i> mahasiswa	4/4	Berhasil
D5	Dosen pembimbing dapat melihat <i>logbook</i> mahasiswa	1. Pengguna (dosen pembimbing) menekan salah satu mahasiswa dari daftar mahasiswa bimbingan	-	Pengguna (dosen pembimbing) dapat melihat <i>logbook</i> termasuk <i>update</i> pengerjaan oleh mahasiswa bimbingan	4/4	Berhasil

Kode	Deskripsi Pengujian	Skenario Pengujian	Input	Hasil yang Diharapkan	Skor Keberhasilan Target Pengguna	Keterangan
D6	Dosen pembimbing dapat melihat riwayat pengajuan bimbingan oleh mahasiswa	<i>Desktop</i> 1. Pengguna (dosen pembimbing) menekan salah satu mahasiswa dari daftar mahasiswa bimbingan 2. Pengguna menekan tombol “Riwayat Pengajuan Bimbingan” <i>Mobile</i> 1. Pengguna menekan tab menu “Riwayat” 2. Pengguna (dosen pembimbing) menekan salah satu mahasiswa dari daftar mahasiswa bimbingan 3. Pengguna menekan tombol “Riwayat Pengajuan Bimbingan”	-	Pengguna (dosen pembimbing) dapat melihat riwayat pengajuan bimbingan oleh mahasiswa bimbingannya	4/4	Berhasil

Kode	Deskripsi Pengujian	Skenario Pengujian	Input	Hasil yang Diharapkan	Skor Keberhasilan Target Pengguna	Keterangan
D7	Mengelola sesi bimbingan berlangsung	1. Pengguna (dosen pembimbing) memulai sesi bimbingan yang akan datang 2. Pengguna memberikan catatan saat sesi sedang berlangsung 3. Pengguna mengakhiri sesi bimbingan	Catatan : “Bab 3 bisa dijelaskan secara lebih detail”	Sesi bimbingan berjalan, catatan yang diberikan pengguna (dosen pembimbing) tersimpan, dan ketika sesi bimbingan diakhiri maka akan masuk ke dalam <i>logbook</i> mahasiswa sesuai dengan masukan yang ditambahkan ketika sesi bimbingan berlangsung	4/4	Berhasil

Kode	Deskripsi Pengujian	Skenario Pengujian	Input	Hasil yang Diharapkan	Skor Keberhasilan Target Pengguna	Keterangan
M1	Mahasiswa dapat mengajukan jadwal bimbingan dengan dosen pembimbing sesuai ketersediaan jadwal dosen pembimbing pilihannya	1. Pengguna (mahasiswa) menekan tombol “Ajukan Bimbingan” pada komponen salah satu dosen pembimbing pada versi <i>Desktop</i> . Untuk versi <i>mobile</i> , pengguna menekan tombol pengajuan bimbingan berwarna hijau di bawah layar 2. Pengguna mengisi jadwal bimbingan sesuai waktu ketersediaan dosen pembimbing terpilih 3. Pengguna menekan tombol “Ajukan Bimbingan” untuk mengirim formulir pengajuan	Tanggal : 8 Mei 2025 Jam: 10:00 Catatan : Finalisasi dokumen	Pengguna (mahasiswa) mendapat konfirmasi berhasil mengirim pengajuan jadwal bimbingan sesuai yang dimasukkan dan dapat melihat komponen “Bimbingan menunggu konfirmasi” tampil di laman dasbor miliknya	4/4	Berhasil

Kode	Deskripsi Pengujian	Skenario Pengujian	Input	Hasil yang Diharapkan	Skor Keberhasilan Target Pengguna	Keterangan
M1	Mahasiswa dapat mengajukan jadwal bimbingan secara manual dengan dosen pembimbing pilihannya	1. Pengguna (mahasiswa) menekan tombol “Ajukan Bimbingan” pada komponen salah satu dosen pembimbing pada versi <i>Desktop</i>. Untuk versi <i>mobile</i>, pengguna menekan tombol pengajuan bimbingan berwarna hijau di bawah layar 2. Pengguna mencentang “Atur jadwal sendiri” kemudian mengisi pilihan tanggal, durasi, dan jam 3. Pengguna menekan tombol “Ajukan Bimbingan” untuk mengirim formulir pengajuan	Tanggal : 8 Mei 2025 Jam: 10:00 Durasi: 30 menit Catatan : Finalisasi dokumen	Pengguna (mahasiswa) mendapat konfirmasi berhasil mengirim pengajuan jadwal bimbingan sesuai yang dimasukkan dan dapat melihat komponen “Bimbingan menunggu konfirmasi” tampil di laman dasbor miliknya	4/4	Berhasil

Kode	Deskripsi Pengujian	Skenario Pengujian	Input	Hasil yang Diharapkan	Skor Keberhasilan Target Pengguna	Keterangan
M2	Mahasiswa menerima notifikasi pengingat untuk mengajukan jadwal bimbingan	1. Pengguna (mahasiswa) mengizinkan pengiriman notifikasi <i>push</i>	Notifikasi terjadwal setiap hari Senin pukul 07:00 WIB	Pengguna (mahasiswa) mendapat notifikasi <i>in-app</i> dan notifikasi <i>push</i> berisi pengingat untuk mengajukan bimbingan pada jam yang ditentukan	4/4	Berhasil
M2	Mahasiswa menerima notifikasi pengingat untuk memperbaiki (menambahkan <i>update</i>) pada <i>logbook</i> miliknya	1. Pengguna (mahasiswa) mengizinkan pengiriman notifikasi <i>push</i>	Notifikasi terjadwal setiap hari Jumat pukul 17:00 WIB	Pengguna (mahasiswa) notifikasi <i>in-app</i> dan notifikasi <i>push</i> berisi pengingat untuk memperbaiki <i>logbook</i> miliknya pada jam yang ditentukan	4/4	Berhasil

Kode	Deskripsi Pengujian	Skenario Pengujian	Input	Hasil yang Diharapkan	Skor Keberhasilan Target Pengguna	Keterangan
M2	Mahasiswa menerima notifikasi jadwal bimbingan yang akan datang	1. Pengguna (mahasiswa) mengizinkan pengiriman notifikasi <i>push</i>	-	Pengguna (mahasiswa) menerima notifikasi <i>in-app</i> dan notifikasi <i>push</i> terkait bimbingan yang akan datang tepat 30 menit sebelum waktu yang telah disetujui	4/4	Berhasil
M2	Mahasiswa menerima notifikasi perubahan status pengajuan bimbingan	1. Pengguna (dosen pembimbing) mengubah status pengajuan bimbingan mahasiswa	Status: Setuju Catatan : -	Pengguna (mahasiswa) menerima notifikasi <i>in-app</i> dan notifikasi <i>push</i> mengenai perubahan status pada pengajuan jadwal bimbingan	4/4	Berhasil
M2	Mahasiswa menerima notifikasi penambahan komentar oleh dosen pembimbing	1. Pengguna (dosen pembimbing) menambahkan komentar di dalam <i>logbook</i> mahasiswa	Komentar: Silakan persiapkan untuk sidang	Pengguna (mahasiswa) menerima notifikasi <i>in-app</i> dan notifikasi <i>push</i> terkait penambahan komentar oleh dosen pembimbing pada <i>logbook</i> miliknya	4/4	Berhasil

Kode	Deskripsi Pengujian	Skenario Pengujian	Input	Hasil yang Diharapkan	Skor Keberhasilan Target Pengguna	Keterangan
M3	Mahasiswa dapat melihat perkembangan pengerjaan tugas akhir miliknya	1. Pengguna (mahasiswa) membuka laman dasbor 2. Pengguna membuka laman <i>logbook</i>	-	Pengguna (mahasiswa) dapat melihat total bimbingan yang telah terlaksana pada dasbor dan melihat <i>logbook</i> miliknya baik yang sudah terisi maupun belum terisi	4/4	Berhasil

Kode	Deskripsi Pengujian	Skenario Pengujian	Input	Hasil yang Diharapkan	Skor Keberhasilan Target Pengguna	Keterangan
M4	Mahasiswa dapat menambahkan <i>Update</i> pada <i>logbook</i> miliknya	<i>Desktop</i> 1. Pengguna (mahasiswa) membuka laman <i>logbook</i> 2. Pengguna mengisi kolom penambahan <i>update</i> dan menambahkan lampiran. 3. Pengguna menekan tombol “Tambah <i>Update</i>” setelah semua kolom terisi <i>Mobile</i> 1. Pengguna (mahasiswa) membuka laman <i>logbook</i> 2. Pengguna menekan tombol “Tambah <i>Update</i>” 3. Pengguna mengisi kolom penambahan <i>update</i> dan menambahkan lampiran.	Konten <i>update</i>: Finalisasi dokumen Lampiran: https://docs.google.com	Pengguna (mahasiswa) mendapat konfirmasi berhasil menambahkan <i>update</i> dan dapat melihat pembaruan pada <i>logbook</i> miliknya	4/4	Berhasil

Kode	Deskripsi Pengujian	Skenario Pengujian	Input	Hasil yang Diharapkan	Skor Keberhasilan Target Pengguna	Keterangan
M4	Mahasiswa dapat menambahkan catatan bimbingan secara manual pada <i>logbook</i> miliknya	1. Pengguna (mahasiswa) membuka laman <i>logbook</i> 2. Pengguna menekan tombol “Catat Bimbingan” 3. Pengguna mengisi catatan bimbingan sesuai dengan siapa dosen yang membimbing, kapan, dan apa yang dibahas 4. Pengguna menekan tombol “Tambah Catatan Bimbingan”	Dosen Pembimbing: Radinal Setyadi nsa, S.Pd, M.T.I. Tanggal : 8 Mei 2025 Jam: 08:00-09:00 Catatan : Memahas implementasi proyek dan persiapan sidang	Pengguna (mahasiswa) mendapat konfirmasi berhasil menambahkan catatan bimbingan secara manual dan dapat melihat pembaruan pada <i>logbook</i> miliknya	4/4	Berhasil

Kode	Deskripsi Pengujian	Skenario Pengujian	Input	Hasil yang Diharapkan	Skor Keberhasilan Target Pengguna	Keterangan
M4	Mahasiswa dapat menghapus <i>update</i> pada <i>logbook</i> bimbingannya	1. Pengguna (mahasiswa) membuka laman <i>logbook</i> 2. Pengguna menekan tombol edit untuk <i>update</i> yang telah dibuat 3. Pengguna menekan tombol “Hapus” dan konfirmasi hapus	-	Pengguna (mahasiswa) mendapat konfirmasi berhasil menghapus <i>update</i> dan melihat pembaruan pada <i>logbook</i> miliknya	4/4	Berhasil
M4	Mahasiswa dapat menghapus catatan bimbingan pada <i>logbook</i> miliknya	1. Pengguna (mahasiswa) membuka laman <i>logbook</i> 2. Pengguna menekan tombol lihat detail untuk catatan bimbingan yang telah dibuat 3. Pengguna menekan tombol “Hapus” dan konfirmasi hapus	-	Pengguna (mahasiswa) mendapat konfirmasi berhasil menghapus catatan bimbingan dan dapat melihat pembaruan pada <i>logbook</i> miliknya	4/4	Berhasil

Kode	Deskripsi Pengujian	Skenario Pengujian	Input	Hasil yang Diharapkan	Skor Keberhasilan Target Pengguna	Keterangan
M4	Mahasiswa dapat mengunduh <i>logbook</i> berisi riwayat bimbingannya	1. Pengguna (mahasiswa) membuka laman <i>logbook</i> 2. Pengguna menekan tombol “Unduh <i>Logbook</i> Bimbingan”	-	Pengguna (mahasiswa) mendapatkan <i>file</i> PDF yang berisikan informasi mengenai riwayat bimbingan yang telah dilakukan	4/4	Berhasil
M5	Mahasiswa dapat melihat riwayat bimbingan yang telah dilakukan	1. Pengguna (mahasiswa) membuka laman dasbor	-	Pengguna (mahasiswa) dapat melihat riwayat bimbingan yang telah diajukan	4/4	Berhasil

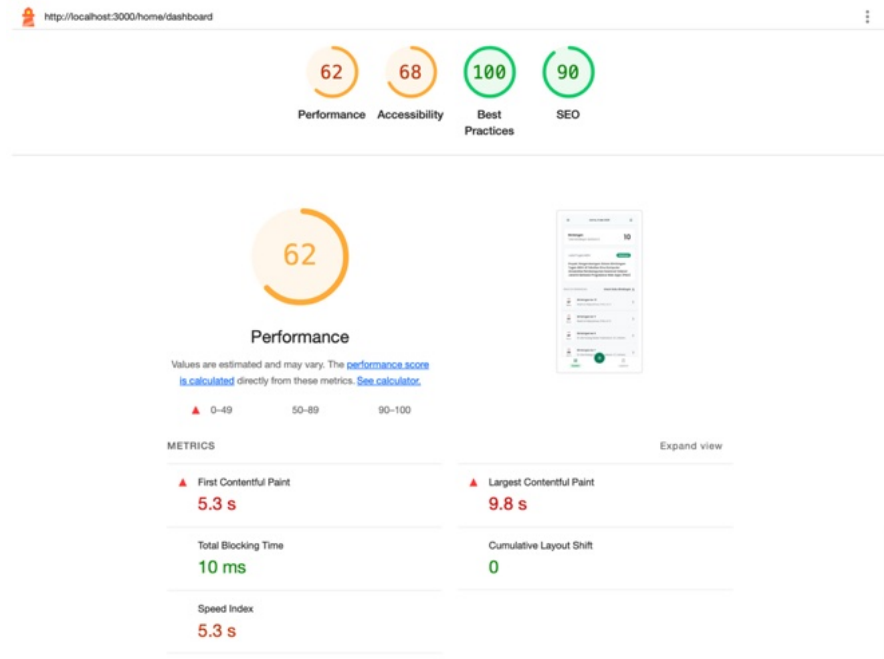
Kode	Deskripsi Pengujian	Skenario Pengujian	Input	Hasil yang Diharapkan	Skor Keberhasilan Target Pengguna	Keterangan
M6	Mengelola sesi bimbingan berlangsung	1. Pengguna (mahasiswa) memulai sesi bimbingan yang akan datang 2. Pengguna memberikan catatan saat sesi sedang berlangsung 3. Pengguna mengakhiri sesi bimbingan	Catatan : “Bab 3 perlu dijelaskan secara lebih detail”	Sesi bimbingan berjalan, catatan yang diberikan pengguna (mahasiswa) tersimpan, dan ketika sesi bimbingan diakhiri maka akan masuk ke dalam <i>logbook</i> mahasiswa sesuai dengan masukan yang ditambahkan ketika sesi bimbingan berlangsung	4/4	Berhasil

Berdasarkan tabel hasil *black box testing*, 29 komponen pengujian telah dilakukan pada berbagai fungsionalitas sistem yang melibatkan aktor dosen pembimbing dan mahasiswa. Pada sisi dosen pembimbing, pengujian mencakup kemampuan untuk mengatur ketersediaan waktu membimbing, menyetujui atau menolak pengajuan jadwal bimbingan, membatalkan jadwal yang telah disetujui, menerima notifikasi terjadwal dan notifikasi terkait *logbook* mahasiswa serta pengajuan bimbingan, memberikan dan menghapus komentar di *logbook*, serta melihat riwayat pengajuan bimbingan.

Sementara itu pada sisi mahasiswa, pengujian difokuskan pada kemampuan untuk mengajukan jadwal bimbingan, menerima berbagai jenis notifikasi (pengingat jadwal, *update logbook*, jadwal bimbingan, perubahan status pengajuan, dan komentar dosen), melihat perkembangan tugas akhir melalui *logbook*, menambahkan *update* dan catatan bimbingan ke *logbook*, menghapus *update* dan catatan, mengunduh *logbook*, serta melihat riwayat bimbingan. Secara keseluruhan, hasil aktual dari pengujian *black box* menunjukkan bahwa seluruh skenario pengujian yang dirancang telah berhasil dieksekusi sesuai dengan hasil yang diharapkan. Ini mengindikasikan bahwa fungsionalitas sistem telah berjalan dengan baik dan memenuhi spesifikasi yang telah ditetapkan. Adapun rata-rata skor keberhasilan dari keseluruhan dari hasil seluruh komponen pengujian *Black Box* adalah 29 dari total 29, sehingga dapat disimpulkan bahwa nilai keberhasilan mencapai 100%.

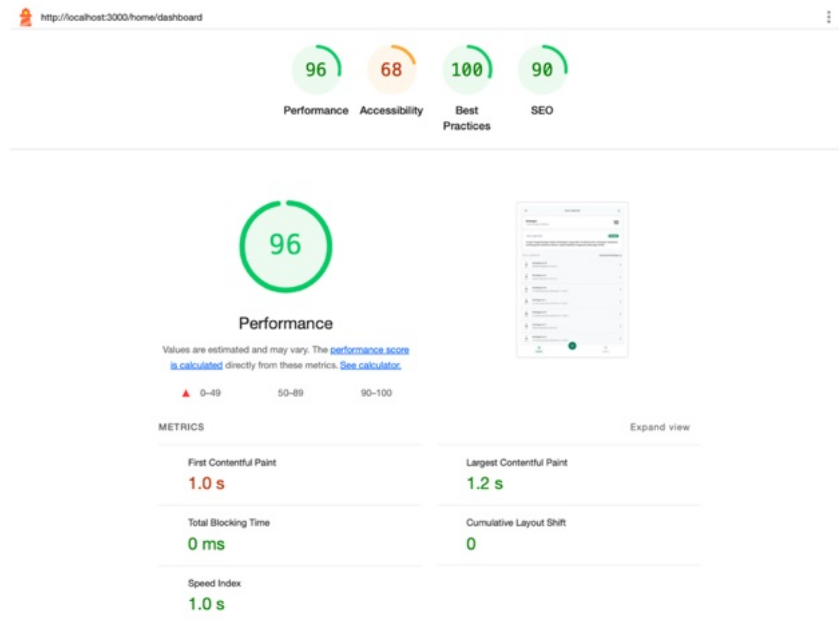
3.5.2. *Lighthouse Testing*

Pengujian *Lighthouse* juga telah dilakukan ketika integrasi dengan sistem yang sebelumnya telah dikembangkan dengan fokus pada metrik nonfungsional meliputi, *Performance*, *Accessibility*, *Best Practice*, *Search Engine Optimized* (SEO), dan kompatibilitas dengan standar *Progressive Web Apps* (PWA).



Gambar 3.28 Hasil Uji *Lighthouse* Versi *Mobile*

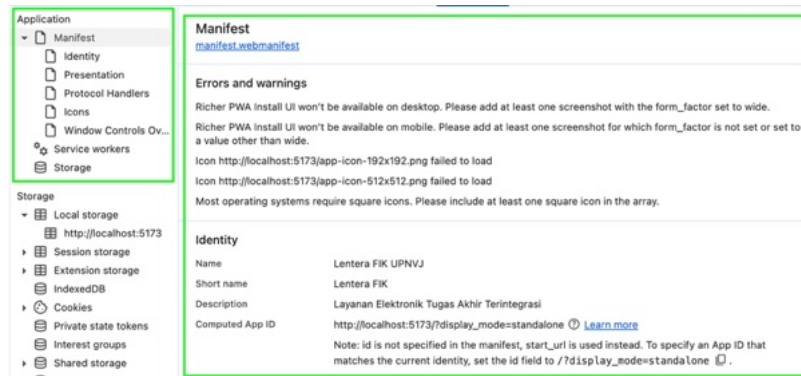
Hasil uji *Lighthouse* pada versi *mobile* menunjukkan skor performa 62/100 atau 62%, yang mengindikasikan masih ada ruang untuk peningkatan dalam hal kinerja. Beberapa metrik kinerja utama menunjukkan area yang perlu dioptimalkan, seperti *First Contentful Paint* (FCP) 5.3 detik dan *Largest Contentful Paint* (LCP) yang cukup tinggi yaitu di angka 9.8 detik. Skor *Accessibility* ada di angka 68/100 atau 68%, menunjukkan perlunya perbaikan dalam aspek aksesibilitas. *Best Practices* dan *SEO* masing-masing mendapat skor 100/100 atau 100% dan 90/100 atau 90%, menunjukkan hasil yang baik di kedua area tersebut.



Gambar 3.29 Hasil Uji *Lighthouse* Versi *Desktop*

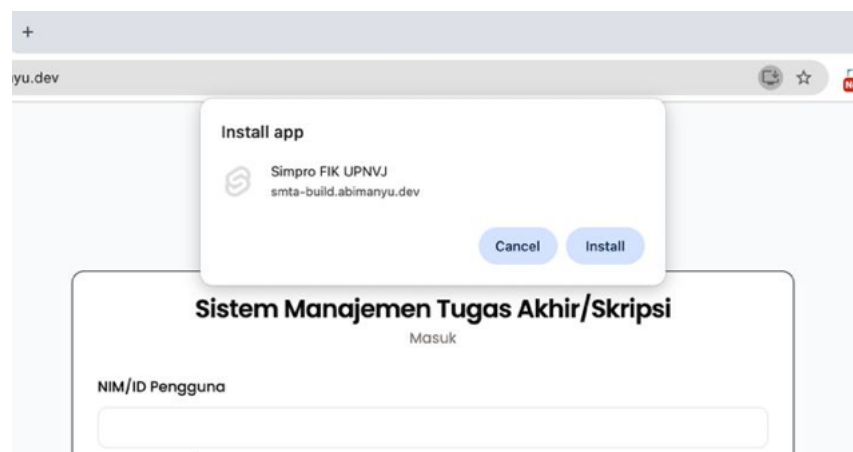
Sementara itu, hasil uji *Lighthouse* pada versi *desktop* menunjukkan performa yang cukup lebih baik dengan skor 96/100 atau 96%. Metrik kinerja utama seperti *First Contentful Paint* (1.0 s), *Largest Contentful Paint* (1.2 s), *Total Blocking Time* (0 ms), dan *Speed Index* (1.0 s). Waktu pemuatan dan interaksi yang cepat ini mendukung pengalaman pengguna yang positif, sehingga secara tidak langsung memvalidasi efektivitas implementasi fungsionalitas yang diuji dalam *black box*. Dengan kata lain, tidak hanya fungsionalitasnya berjalan sesuai kebutuhan, tetapi juga berjalan dengan efisien dan responsif pada versi *desktop*. Namun, pada komponen aksesibilitas nilainya masih sama dengan hasil uji versi *mobile* berada pada angka 68/100 atau 68%.

Adapun hasil pemeriksaan secara manual melalui *Developer Tools* peramban *Chrome* menunjukkan bahwa sistem yang telah diimplementasi saat ini mendukung PWA berdasarkan file “manifest” yang terbaca oleh peramban pada Gambar 3.30.



Gambar 3.30 File Manifest Terbaca oleh Peramban

Selain terbaca oleh peramban, sistem yang dikembangkan juga memungkinkan untuk dipasang ke perangkat pengguna untuk akses lebih mudah melalui layar utama. Adapun kemampuan instalasi aplikasi ditampilkan pada Gambar 3.31 berikut.



Gambar 3.31 Kemampuan Mengunduh Aplikasi Melalui Peramban

3.5.3. User Acceptance Test (UAT)

Tahapan berikutnya dalam pengembangan sistem ini adalah *User Acceptance Test (UAT)*, yang bertujuan untuk memvalidasi apakah fungsionalitas dan alur kerja sistem layanan bimbingan tugas akhir yang dikembangkan telah sesuai dengan kebutuhan dan ekspektasi pengguna. Pengujian ini melibatkan interaksi langsung pengguna dengan sistem untuk mendapatkan umpan balik secara langsung. Pelaksanaan UAT dilakukan secara terpisah untuk dua (2) kelompok pengguna utama. Sesi pertama melibatkan dosen pembimbing dan para pemangku kepentingan di Fakultas Ilmu

Komputer UPNVJ, yang dilaksanakan pada hari Kamis, 22 Mei 2025, pukul 08:30 hingga 09:30 WIB. Partisipan dalam sesi ini mencakup jajaran Wakil Dekan, Koordinator Program Studi Informatika dan Sistem Informasi, dosen penguji, serta perwakilan dari Tim Pengembangan Layanan Teknologi Informasi dan staf Fakultas Ilmu Komputer UPNVJ. Sesi kedua UAT melibatkan partisipasi dari mahasiswa aktif Fakultas Ilmu Komputer UPNVJ untuk memastikan sistem juga memenuhi kebutuhan dari perspektif mahasiswa. Hasil dan analisis dari kedua sesi pengujian penerimaan pengguna ini akan diuraikan lebih lanjut pada tabel berikut.

Tabel 3.6 Hasil *User Acceptance Test* (UAT)

Kode	Deskripsi Pengujian	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual
UAT-1	Dosen pembimbing mengubah jadwal ketersediaan membimbing	1. Pengguna membuka laman dasbor 2. Pengguna menekan tombol pengaturan ketersediaan jadwal membimbing 3. Pengguna menambahkan atau menghapus ketersediaan jadwal membimbing 4. Pengguna menekan tombol simpan	1. Keterangan slot waktu pada laman dasbor berhasil berubah 2. Mahasiswa bimbingan dapat memilih dari jadwal yang telah diatur oleh dosen pembimbing	1. Keterangan slot waktu pada laman dasbor berhasil berubah 2. Mahasiswa bimbingan dapat memilih dari jadwal yang telah diatur oleh dosen pembimbing

Kode	Deskripsi Pengujian	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual
UAT-2	Dosen pembimbing menerima pengajuan jadwal bimbingan	1. Pengguna membuka laman dasbor 2. Pengguna menekan komponen "bimbingan menunggu konfirmasi" 3. Pengguna menekan salah satu pengajuan bimbingan oleh mahasiswa	Pengguna dapat melihat detail pengajuan bimbingan oleh mahasiswa	Pengguna dapat melihat detail pengajuan bimbingan oleh mahasiswa

Kode	Deskripsi Pengujian	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual
UAT-3	Dosen pembimbing menolak pengajuan jadwal bimbingan	1. Pengguna membuka laman dasbor 2. Pengguna menekan komponen "bimbingan menunggu konfirmasi" 3. Pengguna menekan salah satu pengajuan bimbingan oleh mahasiswa 4. Pengguna menekan tombol "Tolak" pada detail pengajuan bimbingan mahasiswa 5. Pengguna mengisi kolom alasan penolakan pada dialog konfirmasi 6. Pengguna menekan tombol "Tolak" pada dialog konfirmasi	1. Jumlah pengajuan bimbingan yang perlu dikonfirmasi pada dasbor dosen pembimbing berkurang 2. Riwayat pengajuan oleh mahasiswa bersangkutan statusnya telah berubah menjadi "Ditolak" 3. Mahasiswa bimbingan yang bersangkutan mendapatkan notifikasi <i>push</i> dan notifikasi <i>in-app</i>	1. Jumlah pengajuan bimbingan yang perlu dikonfirmasi pada dasbor dosen pembimbing berkurang 2. Riwayat pengajuan oleh mahasiswa bersangkutan statusnya telah berubah menjadi "Ditolak" 3. Mahasiswa bimbingan yang bersangkutan mendapatkan notifikasi <i>push</i> dan notifikasi <i>in-app</i>

Kode	Deskripsi Pengujian	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual
UAT-4	Dosen pembimbing menyetujui pengajuan jadwal bimbingan	1. Pengguna membuka laman dasbor 2. Pengguna menekan komponen "bimbingan menunggu konfirmasi" 3. Pengguna menekan salah satu pengajuan bimbingan oleh mahasiswa 4. Pengguna menekan tombol "Setuju" pada detail pengajuan bimbingan mahasiswa 5. Pengguna mengisi kolom catatan tambahan pada dialog konfirmasi 6. Pengguna menekan tombol "Setuju" pada dialog konfirmasi	1. Jumlah pengajuan bimbingan yang perlu dikonfirmasi pada dasbor dosen pembimbing berkurang 2. Riwayat pengajuan oleh mahasiswa bersangkutan statusnya telah berubah menjadi "Disetujui" 3. Terdapat komponen bimbingan yang akan datang pada dasbor mahasiswa dan dosen 4. Mahasiswa bimbingan yang bersangkutan mendapatkan notifikasi <i>push</i> dan notifikasi <i>in-app</i>	1. Jumlah pengajuan bimbingan yang perlu dikonfirmasi pada dasbor dosen pembimbing berkurang 2. Riwayat pengajuan oleh mahasiswa bersangkutan statusnya telah berubah menjadi "Disetujui" 3. Terdapat komponen bimbingan yang akan datang pada dasbor mahasiswa dan dosen 4. Mahasiswa bimbingan yang bersangkutan mendapatkan notifikasi <i>push</i> dan notifikasi <i>in-app</i>

Kode	Deskripsi Pengujian	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual
UAT-5	Dosen pembimbing membatalkan pengajuan bimbingan yang telah disetujui	1. Pengguna membuka laman dasbor 2. Pengguna menekan tombol pembatalan sesi bimbingan yang akan datang 3. Pengguna mengisi kolom alasan penolakan pada dialog konfirmasi 4. Pengguna menekan tombol "Batalkan" pada dialog konfirmasi	1. Jumlah pengajuan bimbingan yang akan datang pada dasbor dosen pembimbing berkurang 2. Riwayat pengajuan oleh mahasiswa bersangkutan statusnya telah berubah menjadi "Dibatalkan" 3. Komponen "Bimbingan yang akan datang" jumlahnya berkurang pada dasbor dosen pembimbing atau mahasiswa 4. Mahasiswa bimbingan yang bersangkutan mendapatkan notifikasi <i>push</i> dan notifikasi <i>in-app</i>	1. Jumlah pengajuan bimbingan yang akan datang pada dasbor dosen pembimbing berkurang 2. Riwayat pengajuan oleh mahasiswa bersangkutan statusnya telah berubah menjadi "Dibatalkan" 3. Komponen "Bimbingan yang akan datang" jumlahnya berkurang pada dasbor dosen pembimbing atau mahasiswa 4. Mahasiswa bimbingan yang bersangkutan mendapatkan notifikasi <i>push</i> dan notifikasi <i>in-app</i>

Kode	Deskripsi Pengujian	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual
UAT-6	Dosen pembimbing menerima notifikasi penambahan <i>update</i> dari mahasiswa bimbingan	1. Pengguna mengaktifkan fitur notifikasi <i>push</i>	1. Pengguna menerima notifikasi <i>push</i> terkait <i>update</i> dari mahasiswa bimbingan 2. Pengguna dapat melihat notifikasi melalui menu notifikasi	1. Pengguna menerima notifikasi <i>push</i> terkait <i>update</i> dari mahasiswa bimbingan 2. Pengguna dapat melihat notifikasi melalui menu notifikasi
UAT-7	Dosen pembimbing menambah komentar di <i>logbook</i> mahasiswa	1. Pengguna membuka laman dasbor 2. Pengguna menekan tombol salah satu mahasiswa dari daftar mahasiswa bimbingan 3. Pengguna menekan tombol "Tambah Komentar" 4. Pengguna mengisi kolom komentar dan menambahkan lampiran tambahan pada dialog penambahan komentar 5. Pengguna menekan tombol "Tambah Komentar" pada dialog penambahan komentar	1. Pengguna melihat komentar yang ditambahkan tampil di dalam <i>Logbook</i> mahasiswa terkait 2. Mahasiswa mendapatkan notifikasi <i>push</i> dan notifikasi <i>in-app</i> terkait penambahan komentar	1. Pengguna melihat komentar yang ditambahkan tampil di dalam <i>Logbook</i> mahasiswa terkait 2. Mahasiswa mendapatkan notifikasi <i>push</i> dan notifikasi <i>in-app</i> terkait penambahan komentar

Kode	Deskripsi Pengujian	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual
UAT-8	Memperbarui komentar milik pengguna di <i>Logbook</i> mahasiswa	1. Pengguna membuka laman dasbor 2. Pengguna menekan tombol salah satu mahasiswa dari daftar mahasiswa bimbingan 3. Pengguna menekan tombol edit pada salah satu komentar yang telah dibuat 4. Pengguna mengubah isi komentar dan menambahkan atau menghapus lampiran pada dialog "Ubah Komentar" 5. Pengguna menekan tombol "Perbarui Komentar" pada dialog "Ubah Komentar"	1. Pengguna melihat komentar yang ada pada <i>Logbook</i> mahasiswa telah berhasil diperbarui 2. Mahasiswa melihat komentar yang diberikan sesuai dengan pembaruan yang dilakukan	1. Pengguna melihat komentar yang ada pada <i>Logbook</i> mahasiswa telah berhasil diperbarui 2. Mahasiswa melihat komentar yang diberikan sesuai dengan pembaruan yang dilakukan

Kode	Deskripsi Pengujian	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual
UAT-9	Dosen pembimbing melihat detail mahasiswa bimbingan	1. Pengguna membuka laman dasbor 2. Pengguna menekan tombol salah satu mahasiswa dari daftar mahasiswa bimbingan	Pengguna dapat melihat informasi mengenai mahasiswa bimbingan termasuk <i>Logbook</i> mahasiswa, deskripsi tugas akhir, dan profil mahasiswa	Pengguna dapat melihat informasi mengenai mahasiswa bimbingan termasuk <i>Logbook</i> mahasiswa, deskripsi tugas akhir, dan profil mahasiswa
UAT-10	Dosen pembimbing menelusuri riwayat pengajuan bimbingan mahasiswa	Desktop 1. Pengguna membuka laman dasbor 2. Pengguna menekan tombol salah satu mahasiswa dari daftar mahasiswa bimbingan 3. Pengguna menekan tombol "Riwayat Pengajuan Bimbingan" Mobile 1. Pengguna membuka tab "Riwayat"	Pengguna dapat melihat informasi mengenai riwayat pengajuan bimbingan yang dilakukan oleh mahasiswa	Pengguna dapat melihat informasi mengenai riwayat pengajuan bimbingan yang dilakukan oleh mahasiswa
UAT-11	Pengingat jadwal bimbingan yang akan datang	1. Pengguna mengaktifkan fitur notifikasi <i>push</i>	Pengguna mendapatkan notifikasi <i>push</i> dan notifikasi <i>in-app</i> terkait jadwal bimbingan yang akan datang	Pengguna mendapatkan notifikasi <i>push</i> dan notifikasi <i>in-app</i> terkait jadwal bimbingan yang akan datang

Kode	Deskripsi Pengujian	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual
UAT-12	Pengingat mengatur ketersediaan jadwal membimbing	1. Pengguna mengaktifkan fitur notifikasi <i>push</i>	Pengguna mendapatkan notifikasi <i>push</i> dan notifikasi <i>in-app</i> terkait pengaturan jadwal ketersediaan membimbing	Pengguna mendapatkan notifikasi <i>push</i> dan notifikasi <i>in-app</i> terkait pengaturan jadwal ketersediaan membimbing
UAT-13	Aplikasi ditambahkan ke layar utama oleh pengguna dosen pembimbing	1. Pengguna menambahkan aplikasi ke layar utama	Pengguna dapat membuka aplikasi melalui <i>shortcut</i> pada layar utama perangkat	Pengguna dapat membuka aplikasi melalui <i>shortcut</i> pada layar utama perangkat

Kode	Deskripsi Pengujian	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual
UAT-14	Mahasiswa dapat mengajukan jadwal bimbingan dengan dosen pembimbing sesuai ketersediaan jadwal dosen pembimbing pilihannya	1. Pengguna (mahasiswa) menekan tombol “Ajukan Bimbingan” pada komponen salah satu dosen pembimbing pada versi <i>Desktop</i>. Untuk versi <i>mobile</i>, pengguna menekan tombol pengajuan bimbingan berwarna hijau di bawah layar 2. Pengguna mengisi jadwal bimbingan sesuai waktu ketersediaan dosen pembimbing terpilih 3. Pengguna menekan tombol “Ajukan Bimbingan” untuk mengirim formulir pengajuan	Pengguna (mahasiswa) mendapat konfirmasi berhasil mengirim pengajuan jadwal bimbingan sesuai yang dimasukkan dan dapat melihat komponen “Bimbingan menunggu dikonfirmasi” tampil di laman dasbor miliknya	Pengguna (mahasiswa) mendapat konfirmasi berhasil mengirim pengajuan jadwal bimbingan sesuai yang dimasukkan dan dapat melihat komponen “Bimbingan menunggu dikonfirmasi” tampil di laman dasbor miliknya

Kode	Deskripsi Pengujian	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual
UAT-15	Mahasiswa dapat mengajukan jadwal bimbingan secara manual dengan dosen pembimbing pilihannya	1. Pengguna (mahasiswa) menekan tombol “Ajukan Bimbingan” pada komponen salah satu dosen pembimbing pada versi <i>Desktop</i>. Untuk versi <i>mobile</i>, pengguna menekan tombol pengajuan bimbingan berwarna hijau di bawah layar 2. Pengguna mencentang “Atur jadwal sendiri” kemudian mengisi pilihan tanggal, durasi, dan jam 3. Pengguna menekan tombol “Ajukan Bimbingan” untuk mengirim formulir pengajuan	Pengguna (mahasiswa) mendapat konfirmasi berhasil mengirim pengajuan jadwal bimbingan sesuai yang dimasukkan dan dapat melihat komponen “Bimbingan menunggu dikonfirmasi” tampil di laman dasbor miliknya	Pengguna (mahasiswa) mendapat konfirmasi berhasil mengirim pengajuan jadwal bimbingan sesuai yang dimasukkan dan dapat melihat komponen “Bimbingan menunggu dikonfirmasi” tampil di laman dasbor miliknya

Kode	Deskripsi Pengujian	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual
UAT-16	Mahasiswa menerima notifikasi pengingat untuk mengajukan jadwal bimbingan	1. Pengguna (mahasiswa) mengizinkan pengiriman notifikasi <i>push</i>	Pengguna (mahasiswa) mendapat notifikasi <i>in-app</i> dan notifikasi <i>push</i> berisi pengingat untuk mengajukan bimbingan pada jam yang ditentukan	Pengguna (mahasiswa) mendapat notifikasi <i>in-app</i> dan notifikasi <i>push</i> berisi pengingat untuk mengajukan bimbingan pada jam yang ditentukan
UAT-17	Mahasiswa menerima notifikasi pengingat untuk memperbarui (menambahkan <i>update</i>) pada <i>logbook</i> miliknya	1. Pengguna (mahasiswa) mengizinkan pengiriman notifikasi <i>push</i>	Pengguna (mahasiswa) notifikasi <i>in-app</i> dan notifikasi <i>push</i> berisi pengingat untuk memperbarui <i>logbook</i> miliknya pada jam yang ditentukan	Pengguna (mahasiswa) notifikasi <i>in-app</i> dan notifikasi <i>push</i> berisi pengingat untuk memperbarui <i>logbook</i> miliknya pada jam yang ditentukan
UAT-18	Mahasiswa menerima notifikasi jadwal bimbingan yang akan datang	1. Pengguna (mahasiswa) mengizinkan pengiriman notifikasi <i>push</i>	Pengguna (mahasiswa) menerima notifikasi <i>in-app</i> dan notifikasi <i>push</i> terkait bimbingan yang akan datang tepat 30 menit sebelum waktu yang telah disetujui	Pengguna (mahasiswa) menerima notifikasi <i>in-app</i> dan notifikasi <i>push</i> terkait bimbingan yang akan datang tepat 30 menit sebelum waktu yang telah disetujui

Kode	Deskripsi Pengujian	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual
UAT-19	Mahasiswa menerima notifikasi perubahan status pengajuan bimbingan	1. Pengguna (mahasiswa) mengizinkan pengiriman notifikasi <i>push</i>	Pengguna (mahasiswa) menerima notifikasi <i>in-app</i> dan notifikasi <i>push</i> mengenai perubahan status pada pengajuan jadwal bimbingan	Pengguna (mahasiswa) menerima notifikasi <i>in-app</i> dan notifikasi <i>push</i> mengenai perubahan status pada pengajuan jadwal bimbingan
UAT-20	Mahasiswa menerima notifikasi penambahan komentar oleh dosen pembimbing	1. Pengguna (mahasiswa) mengizinkan pengiriman notifikasi <i>push</i>	Pengguna (mahasiswa) menerima notifikasi <i>in-app</i> dan notifikasi <i>push</i> terkait penambahan komentar oleh dosen pembimbing pada <i>logbook</i> miliknya	Pengguna (mahasiswa) menerima notifikasi <i>in-app</i> dan notifikasi <i>push</i> terkait penambahan komentar oleh dosen pembimbing pada <i>logbook</i> miliknya

Kode	Deskripsi Pengujian	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual
UAT-21	Mahasiswa dapat menambahkan Update pada <i>logbook</i> miliknya	Desktop - Pengguna (mahasiswa) membuka laman <i>logbook</i> - Pengguna mengisi kolom penambahan <i>update</i> dan menambahkan lampiran. - Pengguna menekan tombol “Tambah Update” setelah semua kolom terisi Mobile - Pengguna (mahasiswa) membuka laman <i>logbook</i> - Pengguna menekan tombol “Tambah Update” - Pengguna mengisi kolom penambahan <i>update</i> dan menambahkan lampiran.	Pengguna (mahasiswa) mendapat konfirmasi berhasil menambahkan <i>update</i> dan dapat melihat pembaruan pada <i>logbook</i> miliknya	Pengguna (mahasiswa) mendapat konfirmasi berhasil menambahkan <i>update</i> dan dapat melihat pembaruan pada <i>logbook</i> miliknya

Kode	Deskripsi Pengujian	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual
UAT-22	Mahasiswa dapat melihat perkembangan pengerjaan tugas akhir miliknya	1. Pengguna (mahasiswa) membuka laman dasbor 2. Pengguna membuka laman <i>logbook</i>	Pengguna (mahasiswa) dapat melihat total bimbingan yang telah terlaksana pada dasbor dan melihat <i>logbook</i> miliknya baik yang sudah terisi maupun belum terisi	Pengguna (mahasiswa) dapat melihat total bimbingan yang telah terlaksana pada dasbor dan melihat <i>logbook</i> miliknya baik yang sudah terisi maupun belum terisi
UAT-23	Mahasiswa dapat menambahkan catatan bimbingan secara manual pada <i>logbook</i> miliknya	1. Pengguna (mahasiswa) membuka laman <i>logbook</i> 2. Pengguna menekan tombol “Catat Bimbingan” 3. Pengguna mengisi catatan bimbingan sesuai dengan siapa dosen yang membimbing, kapan, dan apa yang dibahas 4. Pengguna menekan tombol “Tambah Catatan Bimbingan”	Pengguna (mahasiswa) mendapat konfirmasi berhasil menambahkan catatan bimbingan secara manual dan dapat melihat pembaruan pada <i>logbook</i> miliknya	Pengguna (mahasiswa) mendapat konfirmasi berhasil menambahkan catatan bimbingan secara manual dan dapat melihat pembaruan pada <i>logbook</i> miliknya

Kode	Deskripsi Pengujian	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual
UAT-24	Mahasiswa dapat menghapus <i>update</i> pada <i>logbook</i> bimbingannya	1. Pengguna (mahasiswa) membuka laman <i>logbook</i> 2. Pengguna menekan tombol edit untuk <i>update</i> yang telah dibuat 3. Pengguna menekan tombol “Hapus” dan konfirmasi hapus	Pengguna (mahasiswa) mendapat konfirmasi berhasil menghapus <i>update</i> dan melihat pembaruan pada <i>logbook</i> miliknya	Pengguna (mahasiswa) mendapat konfirmasi berhasil menghapus <i>update</i> dan melihat pembaruan pada <i>logbook</i> miliknya
UAT-25	Mahasiswa dapat menghapus catatan bimbingan pada <i>logbook</i> miliknya	1. Pengguna (mahasiswa) membuka laman <i>logbook</i> 2. Pengguna menekan tombol lihat detail untuk catatan bimbingan yang telah dibuat 3. Pengguna menekan tombol “Hapus” dan konfirmasi hapus	Pengguna (mahasiswa) mendapat konfirmasi berhasil menghapus catatan bimbingan dan dapat melihat pembaruan pada <i>logbook</i> miliknya	Pengguna (mahasiswa) mendapat konfirmasi berhasil menghapus catatan bimbingan dan dapat melihat pembaruan pada <i>logbook</i> miliknya
UAT-26	Mahasiswa dapat mengunduh <i>logbook</i> berisi riwayat bimbingannya	1. Pengguna (mahasiswa) membuka laman <i>logbook</i> 2. Pengguna menekan tombol “Unduh Logbook Bimbingan”	Pengguna (mahasiswa) mendapatkan <i>file</i> PDF yang berisikan informasi mengenai riwayat bimbingan yang telah dilakukan	Pengguna (mahasiswa) mendapatkan <i>file</i> PDF yang berisikan informasi mengenai riwayat bimbingan yang telah dilakukan

Kode	Deskripsi Pengujian	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual
UAT-27	Mahasiswa dapat melihat riwayat bimbingan yang telah dilakukan	1. Pengguna (mahasiswa) membuka laman dasbor	Pengguna (mahasiswa) dapat melihat riwayat bimbingan yang telah diajukan	Pengguna (mahasiswa) dapat melihat riwayat bimbingan yang telah diajukan
UAT-28	Mengelola sesi bimbingan berlangsung	1. Pengguna (mahasiswa) memulai sesi bimbingan yang akan datang 2. Pengguna memberikan catatan saat sesi sedang berlangsung 3. Pengguna mengakhiri sesi bimbingan	Sesi bimbingan berjalan, catatan yang diberikan pengguna (mahasiswa) tersimpan, dan ketika sesi bimbingan diakhiri maka akan masuk ke dalam <i>logbook</i> mahasiswa sesuai dengan masukan yang ditambahkan ketika sesi bimbingan berlangsung	Sesi bimbingan berjalan, catatan yang diberikan pengguna (mahasiswa) tersimpan, dan ketika sesi bimbingan diakhiri maka akan masuk ke dalam <i>logbook</i> mahasiswa sesuai dengan masukan yang ditambahkan ketika sesi bimbingan berlangsung
UAT-29	Aplikasi ditambahkan ke layar utama oleh pengguna mahasiswa	1. Pengguna menambahkan aplikasi ke layar utama	Pengguna dapat membuka aplikasi melalui <i>shortcut</i> pada layar utama perangkat	Pengguna dapat membuka aplikasi melalui <i>shortcut</i> pada layar utama perangkat

Berdasarkan hasil pelaksanaan UAT dari tabel Tabel 3.6 di atas yang melibatkan partisipasi dari dosen pembimbing serta pemangku kepentingan Fakultas Ilmu Komputer UPNVJ dan juga mahasiswa aktif, dapat disimpulkan bahwa sistem bimbingan tugas akhir yang dikembangkan telah diterima dengan baik. Seluruh skenario pengujian dari UAT-1 hingga UAT-29, yang mencakup

fungsionalitas inti seperti manajemen jadwal ketersediaan dosen, proses pengajuan, persetujuan, penolakan, dan pembatalan sesi bimbingan oleh dosen pembimbing, penerimaan notifikasi oleh dosen dan mahasiswa untuk berbagai aktivitas, pengelolaan *logbook* oleh dosen dan mahasiswa, hingga fitur instalasi aplikasi PWA, menunjukkan bahwa hasil aktual yang diperoleh sepenuhnya sesuai dengan hasil yang diharapkan oleh pengguna. Hal ini mengindikasikan bahwa sistem yang dikembangkan telah berhasil memenuhi kebutuhan fungsional dan dapat mendukung alur proses bimbingan tugas akhir secara efektif dari perspektif kedua kelompok pengguna. Adapun tiga (3) masukan yang disampaikan oleh target pengguna kepada penulis ketika melaksanakan UAT diuraikan pada Tabel 3.7 berikut.

Tabel 3.7 Masukkan Pengguna Terkait Skenario Uji *User Acceptance Test* (UAT)

No.	Jenis Pengguna	Nama	Masukkan
1.	Dosen Pembimbing	Dr. Bambang Saras Yulistiawan, S.T., M.Kom	Buku bimbingan perlu dibuat supaya mendukung tanda tangan digital untuk mengurangi proses pencetakan menggunakan kertas dan memaksimalkan fungsi buku bimbingan secara digital.
2.	Dosen Pembimbing	Neny Rosmawarni, M.Kom	Tambahkan Kop Surat Fakultas Ilmu Komputer UPNVJ pada hasil unduhan buku bimbingan mahasiswa, karena ini bentuk dokumen yang dikeluarkan oleh fakultas.
3.	Dosen Pembimbing	Novi Trisman Hadi, S.Pd., M.Kom.	Buat batas minimal jumlah bimbingan untuk mendaftar seminar proposal dan sidang akhir menjadi dinamis karena kebijakan mengenai persyaratan tersebut dapat berubah sewaktu-waktu.

3.5.4. Implementasi Hasil *User Acceptance Test* (UAT)

Berdasarkan masukan yang disampaikan oleh target pengguna dosen pembimbing selama pelaksanaan *User Acceptance Test* (UAT), beberapa tindak lanjut dilakukan guna menyempurnakan sistem yang dikembangkan dalam lingkup proyek ini. Pertama, terkait dukungan tanda tangan digital pada buku bimbingan guna mengurangi penggunaan kertas dan memaksimalkan fungsi digital, hal ini sejalan dengan fitur yang telah dikembangkan. Kedua, usulan untuk menambahkan kop surat Fakultas Ilmu Komputer UPNVJ pada hasil unduhan buku bimbingan mahasiswa untuk meningkatkan aspek formalitas dokumen resmi yang dikeluarkan oleh fakultas. Terakhir, menanggapi saran agar batas minimal jumlah bimbingan untuk pendaftaran seminar proposal dan sidang akhir dibuat dinamis sehingga persyaratan tersebut dapat diatur secara fleksibel untuk mengantisipasi perubahan kebijakan sewaktu-waktu.

a. Identifikasi Kebutuhan Fungsional

Beberapa kebutuhan fungsional baru diidentifikasi untuk mengakomodasi fitur-fitur lanjutan, terutama terkait proses tanda tangan digital dan pengelolaan syarat minimal bimbingan. Di antara penambahan tersebut, terdapat kebutuhan fungsional untuk aktor "Admin" yang beririsan dengan kebutuhan tanda tangan digital pada aktor "Dosen Pembimbing". Meskipun pengembangan peran "Admin" secara spesifik berada di luar lingkup proyek ini, kebutuhan ini tetap penting untuk dipertimbangkan demi kelengkapan dan keberlanjutan sistem secara keseluruhan. Analisis terkait pengembangan lanjutan berdasarkan kebutuhan fungsional diuraikan pada Tabel 3.8 berikut.

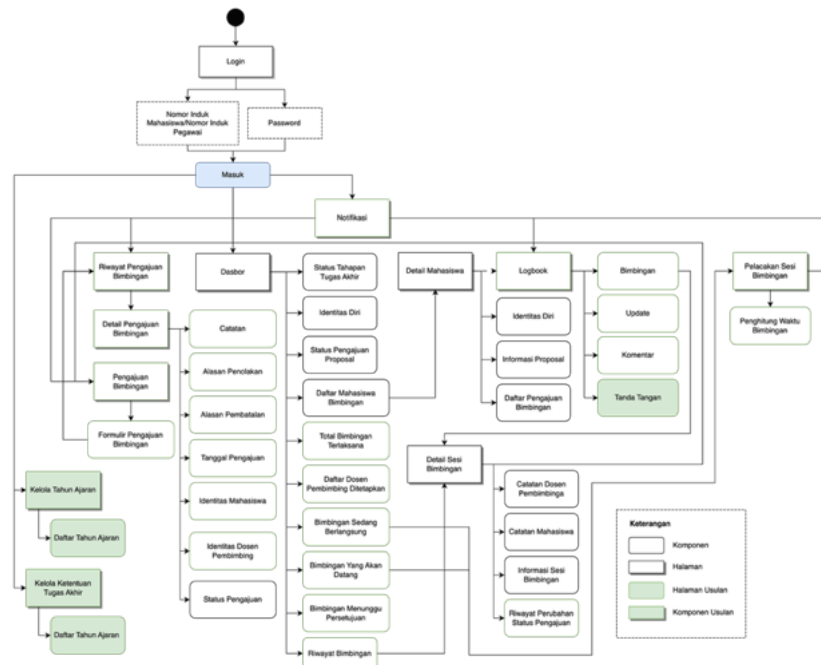
Tabel 3.8 Penambahan Kebutuhan Fungsional

Kode	Jenis Pengguna	Kebutuhan Fungsional	Deskripsi
D8	Dosen Pembimbing	Tanda Tangan Digital pada Buku Bimbingan Mahasiswa	Dosen pembimbing dapat melakukan verifikasi dan menandatangani buku bimbingan mahasiswa sesuai dengan jumlah minimal bimbingan berdasarkan pedoman tugas akhir
M7	Mahasiswa	Unduh Buku Bimbingan yang Sudah Ditandatangani	Mahasiswa dapat mengunduh buku bimbingan saat telah disetujui oleh seluruh dosen pembimbing.
A1	Admin	Pengaturan Jumlah Minimal Bimbingan Sebagai Syarat Persetujuan Mendaftar Seminar Proposal atau Sidang Akhir	Admin dapat mengelola pengaturan jumlah minimal bimbingan sesuai dengan pedoman tugas akhir untuk dapat ditandatangani oleh dosen pembimbing dan mendaftar seminar proposal atau sidang akhir.

b. Perubahan Arsitektur Informasi

Menindaklanjuti hasil identifikasi kebutuhan fungsional berdasarkan umpan balik dari UAT, dilakukan penyesuaian pada Arsitektur Informasi sistem untuk mengintegrasikan fungsionalitas baru yang telah dikembangkan. Perubahan ini bertujuan untuk menyusun struktur navigasi dan alur informasi yang dapat mengakomodasi fitur-fitur lanjutan, seperti tanda tangan digital dan pengelolaan dinamis untuk ketentuan tugas akhir. Pembaruan arsitektur ini memastikan bahwa fitur baru dapat diakses secara logis oleh pengguna yang relevan dan

terintegrasi dengan baik ke dalam alur sistem yang sudah ada. Visualisasi mengenai perubahan arsitektur informasi disajikan pada Gambar 3.32 di bawah.

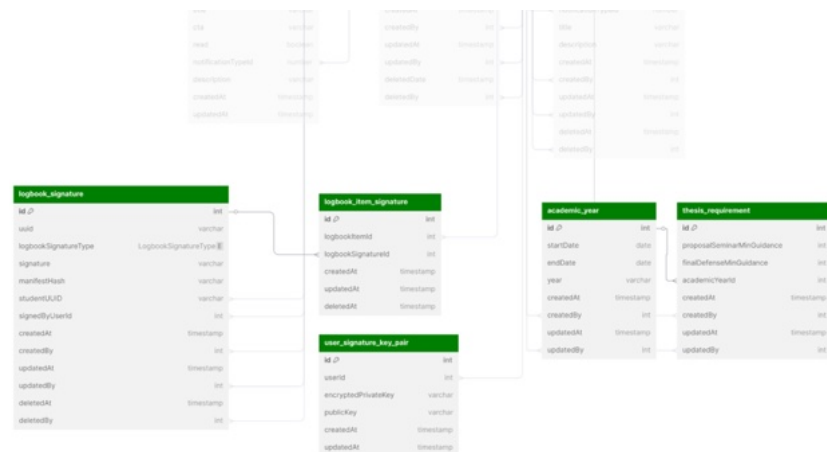


Gambar 3.32 Perubahan Arsitektur Informasi Sesuai dengan Lingkup Pengembangan Proyek

Perubahan utama pada arsitektur informasi yang ditandai dengan warna hijau pada Gambar 3.32, meliputi penambahan beberapa halaman dan komponen usulan baru. Untuk mendukung fitur tanda tangan digital, ditambahkan komponen usulan “Tanda Tangan” dalam alur 'Logbook', yang dapat diakses dosen pembimbing untuk melakukan verifikasi dan penandatanganan secara digital. Selain itu, untuk mengakomodasi pengelolaan ketentuan tugas akhir yang dinamis, diusulkan pula penambahan halaman baru yaitu “Kelola Tahun Ajaran” dan “Kelola Ketentuan Tugas Akhir”, yang dapat diakses oleh pengguna dengan hak akses administratif untuk mengatur syarat minimal bimbingan sesuai tahun ajaran yang berlaku.

c. Perubahan *Entity Relationship Diagram* (ERD)

Berdasarkan identifikasi kebutuhan fungsional hasil umpan balik pada tahap UAT, dilakukan serangkaian penyesuaian dan penambahan pada skema *Entity Relationship Diagram* (ERD) sistem yang disesuaikan dengan perubahan arsitektur informasi. Perubahan ini dilakukan untuk mengakomodasi implementasi fungsionalitas lanjutan yang diidentifikasi dari masukan pengguna, terutama untuk mendukung mekanisme tanda tangan digital yang lebih aman dan pengelolaan persyaratan bimbingan yang dinamis. Penyesuaian skema ini memastikan bahwa struktur basis data memiliki integritas dan mampu menyimpan serta mengelola relasi data yang lebih kompleks untuk fitur-fitur tersebut.



Gambar 3.33 Perubahan pada Entity Relationship Diagram (ERD)

Berdasarkan diagram pada Gambar 3.33 di atas, perubahan utama meliputi penambahan beberapa tabel baru yang ditandai dengan warna hijau untuk mendukung fitur baru hasil umpan balik UAT. Tabel “academic_year” dan “thesis_requirement” ditambahkan untuk memungkinkan pengelolaan syarat minimal bimbingan yang dinamis sesuai tahun ajaran. Adapun untuk mendukung fitur tanda tangan digital, ditambahkan tabel “logbook_signature” yang berfungsi untuk menyimpan catatan setiap peristiwa penandatanganan, dan tabel

“logbook_item_signature” sebagai tabel relasi yang menghubungkan setiap tanda tangan dengan komponen riwayat bimbingan spesifik. Untuk mendukung keamanan proses, ditambahkan tabel “user_signature_key_pair” yang berperan dalam menyimpan pasangan kunci publik dan kunci privat terenkripsi milik pengguna sebagai dasar dalam proses verifikasi tanda tangan digital.

d. Implementasi *Back-end*

i. Migrasi Basis Data

Untuk mengakomodasi umpan balik serta finalisasi fitur pada proyek ini, dilakukan beberapa penyesuaian lanjutan pada skema basis data. Migrasi ini bertujuan untuk memastikan struktur data dapat secara optimal mendukung fungsionalitas yang dikembangkan lebih lanjut, termasuk proses tanda tangan digital dan pengelolaan persyaratan tugas akhir yang dinamis sesuai tahun ajaran. Perubahan tersebut meliputi penambahan tabel-tabel baru yang krusial untuk fitur tersebut serta modifikasi pada tabel yang sudah ada seperti penambahan indeks unik untuk meningkatkan integritas data. Deskripsi mengenai migrasi yang dilakukan diuraikan pada Tabel 3.9 di bawah.

Tabel 3.9 Migrasi Basis Data Lanjutan Setelah *User Acceptance Test* (UAT)

No.	Nama File	Deskripsi Migrasi
1.	20250518034103-add-unique-constraint-user.cjs	Penambahan indeks unik untuk kolom “uuid” pada tabel “users”
2.	20250518035500-create-logbook-signature.cjs	Penambahan tabel baru untuk menyimpan riwayat tanda tangan digital pada buku bimbingan mahasiswa
3.	20250518044539-create-logbook-item-signature.cjs	Penambahan tabel baru untuk menyimpan relasi antara riwayat bimbingan mahasiswa

		dengan tanda tangan digital oleh dosen pembimbing
4.	20250530155215-create-academic-year.cjs	Penambahan tabel baru untuk menyimpan data tahun ajaran
5.	20250530155220-create-thesis-requirement.cjs	Penambahan tabel baru untuk menyimpan data persyaratan atau pedoman tugas akhir yang berlaku pada tahun ajaran tertentu
6.	20250609104012-create-users-signature-key-pair.cjs	Penambahan tabel baru untuk menyimpan data <i>private key</i> terenkripsi dan <i>public key</i> milik pengguna yang digunakan untuk melakukan tanda tangan secara digital.

ii. Penambahan Jenis Notifikasi

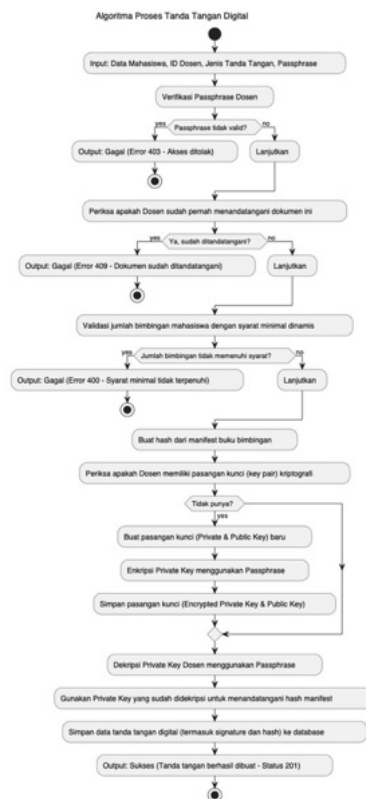
Terkait fitur yang telah dikembangkan sebelumnya yang memungkinkan penyampaian informasi penting secara otomatis kepada pengguna, dilakukan penambahan jenis notifikasi baru ke dalam sistem terkait kebutuhan tambahan mengenai tanda tangan digital. Notifikasi tambahan ini dirancang untuk memberikan pembaruan status secara langsung kepada mahasiswa terkait proses formal penyelesaian bimbingan tugas akhir. Jenis interaksi dan target pengiriman notifikasi diuraikan pada Tabel 3.10 berikut.

Tabel 3.10 Penambahan Jenis Notifikasi

No.	Jenis Notifikasi	Jenis Interaksi	Target Pengiriman
1.	LOGBOOK_SIGNED	Ketika dosen pembimbing menandatangani buku bimbingan mahasiswa	Mahasiswa

iii. Algoritma Tanda Tangan Digital

Pengembangan lanjutan ini memungkinkan mahasiswa untuk mengunduh buku bimbingan yang telah disetujui secara resmi, yang merupakan salah satu syarat penting untuk mendaftar seminar proposal atau sidang akhir. Proses tanda tangan digital ini dirancang untuk memastikan bahwa mahasiswa telah memenuhi jumlah minimal sesi bimbingan yang ditetapkan secara dinamis sesuai dengan kebijakan akademik yang berlaku, sekaligus menjaga integritas data bimbingan. Alur kerja algoritma divisualisasikan pada Gambar 3.34 di bawah.



Gambar 3.34 Visualisasi Algoritma Proses Tanda Tangan Digital

Berdasarkan Gambar 3.34 di atas, proses diawali dengan otorisasi pengguna melalui verifikasi *passphrase* atau dalam konteks proyek ini merupakan hal yang sama dengan *password* akun pengguna. Kemudian dilanjutkan

dengan serangkaian validasi seperti pengecekan duplikasi tanda tangan dan pemenuhan syarat minimal bimbingan yang dinamis. Jika semua validasi berhasil, sistem akan membuat sebuah *hash* dari *manifest* buku bimbingan untuk menjaga integritas data. Langkah selanjutnya adalah manajemen kunci kriptografi, di mana sistem akan memeriksa keberadaan pasangan kunci milik dosen pembimbing; jika belum ada, sistem akan membuat dan mengenkripsi kunci privat dengan *passphrase* dosen untuk memastikan keamanan data kunci privat. Terakhir, kunci privat yang telah didekripsi digunakan untuk menandatangani *hash manifest*, dan bukti tanda tangan digital ini disimpan ke dalam basis data sebagai rekaman verifikasi yang sah.

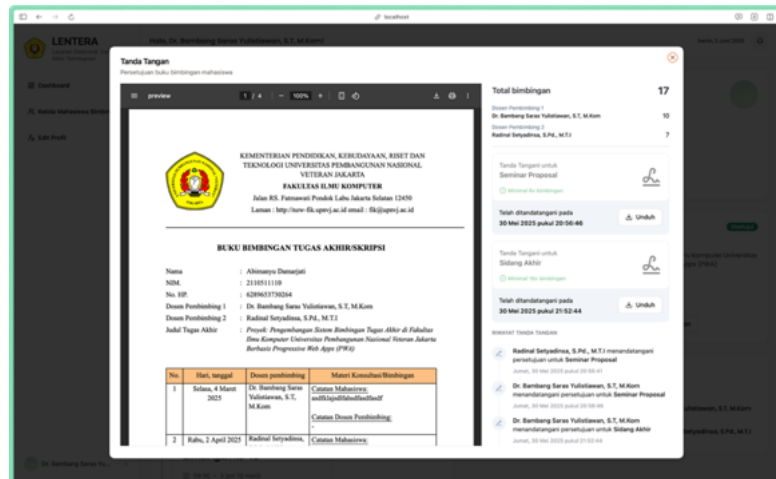
Berkaitan dengan proses pembuatan nilai *hash*, algoritma kriptografis yang digunakan merupakan SHA-256 (*Secure Hash Algorithm* 256-bit). Algoritma tersebut merupakan fungsi satu arah yang menghasilkan nilai *hash* dengan panjang tetap 256 bit (direpresentasikan sebagai 64 karakter heksadesimal) yang unik untuk setiap set data masukan. Hal ini membuat jika ada perubahan sekecil apapun pada data *manifest*, maka nilai *hash* yang dihasilkan akan berubah total, sehingga memudahkan pendeteksian modifikasi yang tidak sah. SHA-256 memiliki sifat yang deterministik, yaitu masukan yang sama selalu menghasilkan keluaran yang sama serta tidak akan menemukan dua masukan berbeda dengan *hash* yang sama sehingga membuat algoritma ini cocok serta sesuai dengan standar industri untuk menciptakan 'sidik jari' digital dari *manifest* buku bimbingan (National Institute of Standards and Technology 2023). 'Sidik jari' digital inilah yang kemudian diproses lebih lanjut menggunakan Kriptografi Kurva Elips (ECC), di mana *hash* yang dihasilkan akan ditandatangani secara matematis

menggunakan kunci privat milik dosen untuk menghasilkan sebuah tanda tangan digital yang unik. Pada tahap verifikasi, tanda tangan ini dapat divalidasi keasliannya oleh siapa pun menggunakan kunci publik dosen yang bersangkutan, yang membuktikan bahwa dokumen tidak berubah dan benar-benar ditandatangani oleh dosen tersebut. Penggunaan kombinasi algoritma ini memastikan bahwa data yang ditandatangani oleh dosen pembimbing merupakan data yang sama persis dengan yang telah diverifikasi, sehingga menjaga keutuhan informasi selama proses bimbingan hingga pendaftaran seminar proposal atau sidang akhir.

e. Implementasi *Front-end*

i. Tanda Tangan Buku Bimbingan Digital

Seperti yang ditampilkan pada Gambar 3.35 di bawah, pengguna dosen dapat melakukan verifikasi dan menandatangani secara digital buku bimbingan mahasiswa. Dosen pembimbing dapat memeriksa secara langsung buku bimbingan terkini yang telah dimasukkan oleh mahasiswa melalui tampilan dokumen pada sebelah kiri dialog. Selain itu, dosen pembimbing dapat menandatangani sesuai kebutuhan baik untuk verifikasi memenuhi persyaratan seminar proposal maupun sidang akhir. Adapun penambahan kop surat yang menyatakan bahwa buku bimbingan ini merupakan hasil cetakan Fakultas Ilmu Komputer UPNVJ.



Gambar 3.35 Tampilan Tanda Tangan Digital oleh Dosen Pembimbing

ii. Pengaturan Ketentuan Jumlah Minimal Bimbingan oleh Pengguna Admin

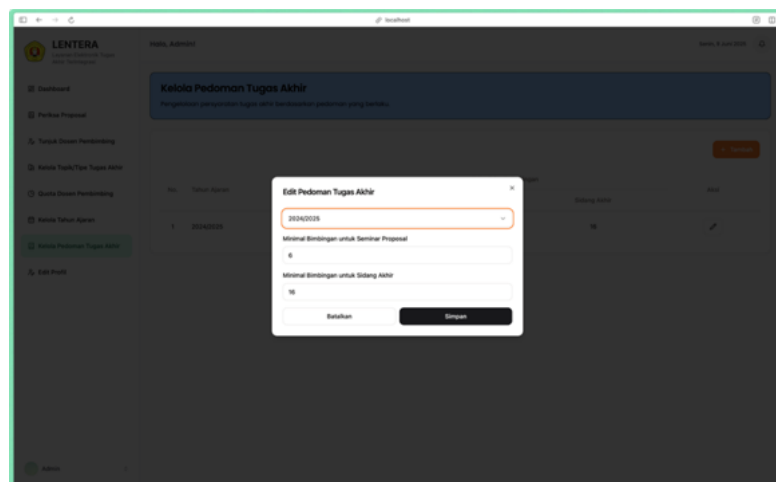
Terkait kebutuhan fungsional hasil umpan balik UAT mengenai syarat minimal bimbingan dinamis, terdapat kebutuhan baru dengan pengguna di luar lingkup proyek yang terikat dengan pengembangannya. Berdasarkan gambar Gambar 3.36 di bawah, pengaturan ketentuan jumlah minimal bimbingan disimpan dalam tabel ketentuan pedoman tugas akhir yang terikat dengan periode tahun ajaran tertentu. Hal ini dilakukan untuk memastikan bahwa ketentuan pedoman tugas akhir diatur sesuai dengan periode tahun ajaran berjalan ketika mahasiswa sedang melakukan proses pengerjaan tugas akhir. Komponen yang dapat diubah pada pengembangan pertama dari fitur ini merupakan jumlah

minimal bimbingan untuk mendaftar sidang proposal dan jumlah minimal bimbingan untuk mendaftar sidang akhir.



Gambar 3.36 Tampilan Daftar Ketentuan Pedoman Tugas Akhir

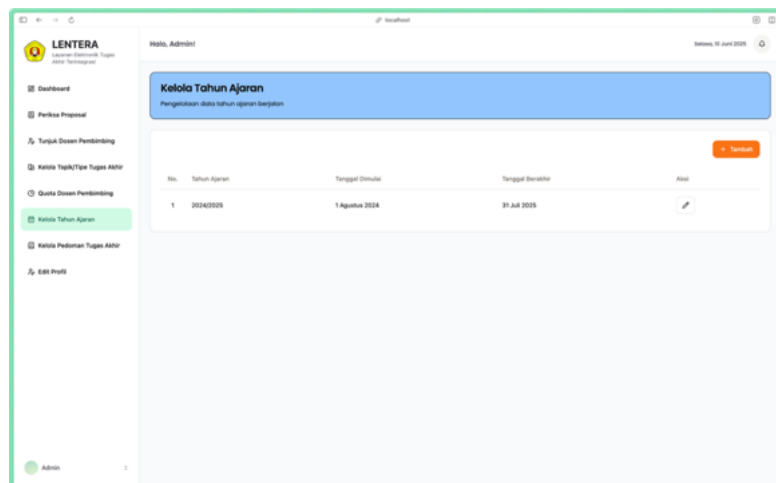
Adapun berdasarkan Gambar 3.37 di bawah, tampilan edit atau tambahkan pedoman tugas akhir disajikan dalam bentuk *pop-up* yang memungkinkan pengguna memilih tahun ajaran yang sudah dibuat pada menu Kelola Tahun Ajaran. Selain itu, terdapat *input* yang dapat menerima angka terkait jumlah aturan dari jumlah minimal bimbingan baik untuk seminar proposal maupun sidang akhir.



Gambar 3.37 Tampilan Edit atau Menambahkan Ketentuan Pedoman Tugas Akhir Berdasarkan Periode Tahun Ajaran Tertentu

iii. Pengaturan Tahun Ajaran oleh Pengguna Admin

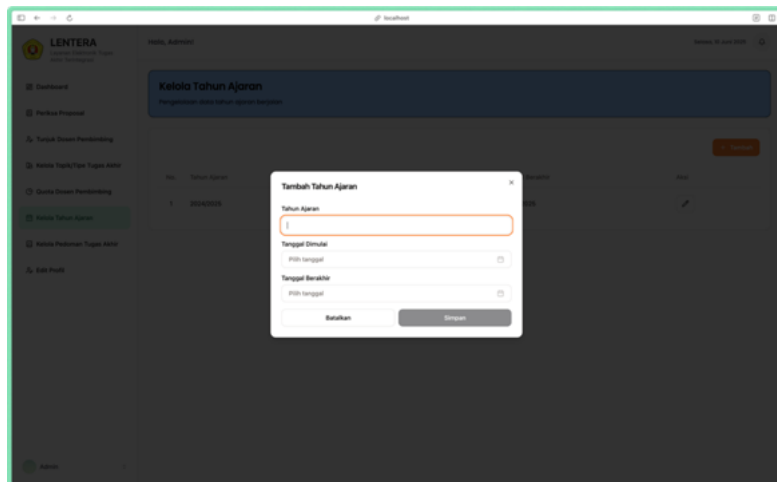
Sesuai dengan kebutuhan untuk pengaturan ketentuan minimal jumlah bimbingan yang dikembangkan, normalisasi fitur dilakukan dengan membuat relasi ketentuan tersebut dengan tahun ajaran yang sedang berjalan. Hal ini memungkinkan perubahan yang dinamis mengenai syarat minimal jumlah bimbingan untuk mendaftar seminar proposal maupun sidang akhir pada ketentuan yang berlaku selama periode saat ini. Tampilan antarmuka terkait laman untuk mengelola tahun ajaran oleh pengguna admin divisualisasikan pada Gambar 3.38 di bawah.



Gambar 3.38 Tampilan Laman Kelola Tahun Ajaran

Terkait dengan data tahun ajaran, Gambar 3.39 di bawah menggambarkan tampilan untuk melakukan tambah atau edit tahun ajaran tertentu. Formulir untuk perubahan atau penambahan ini memungkinkan admin untuk memberikan nama tahun ajaran serta tanggal dimulai dan diakhirinya tahun ajaran tertentu. Dengan demikian, data tahun ajaran dapat dimanfaatkan lebih lanjut untuk kebutuhan lain yang

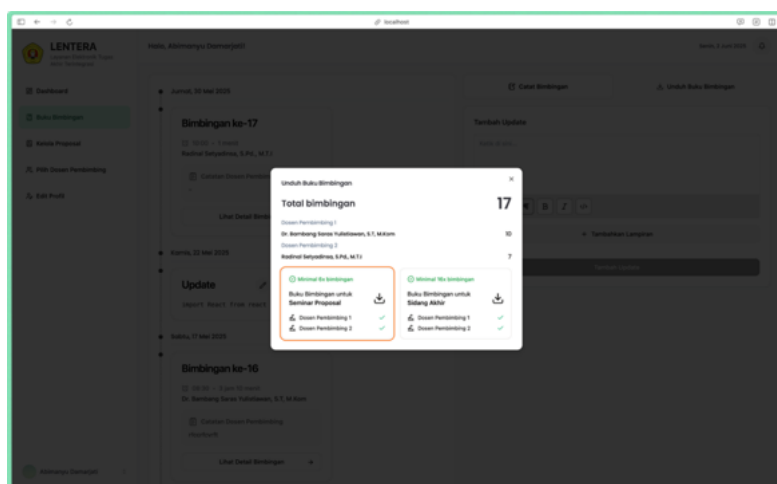
terkait dengan proses administrasi mahasiswa pada tahun ajaran berjalan.



Gambar 3.39 Tampilan Edit atau Tambah Tahun Ajaran

iv. Unduh Buku Bimbingan

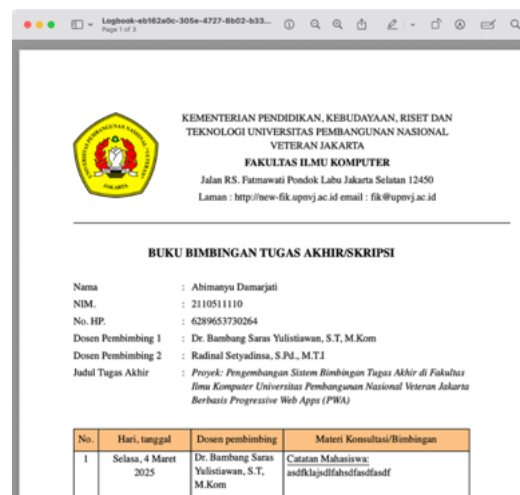
Gambar 3.40 di bawah menampilkan dialog dari proses unduh buku bimbingan dari sisi pengguna mahasiswa. Mahasiswa dapat mengunduh buku bimbingan ketika minimal jumlah bimbingan telah terpenuhi dan seluruh dosen pembimbing telah melakukan verifikasi dan tanda tangan secara digital. Terdapat dua opsi untuk mengunduh sesuai kebutuhan sebagai salah satu persyaratan mendaftar seminar proposal atau sidang akhir.



Gambar 3.40 Tampilan Unduh Buku Bimbingan oleh Mahasiswa

v. Hasil *Printable Document Format* (PDF) Buku Bimbingan Mahasiswa

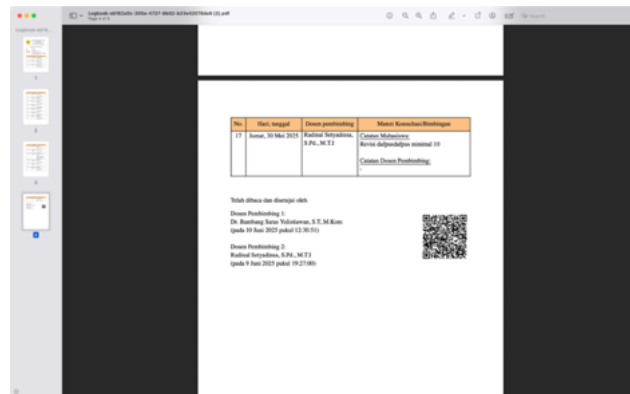
Gambar 3.41 di bawah menampilkan hasil dari pengunduhan *logbook* mahasiswa, yang berisikan informasi mengenai mahasiswa dan tugas akhirnya, serta riwayat-riwayat bimbingan yang telah dilakukan. Struktur dokumen yang dihasilkan mengacu pada Pedoman Tugas Akhir FIK Tahun 2024.... Dokumen buku bimbingan ini juga dilengkapi dengan hasil verifikasi oleh seluruh dosen pembimbing ditandai dengan catatan waktu kapan dosen pembimbing melakukan verifikasi atau tanda tangan digital disertai dengan kode *Quick Response* (QR) yang dapat dipindai untuk melakukan verifikasi lebih lanjut terkait dokumen tersebut. Dengan demikian, dokumen hasil pengunduhan ini dapat digunakan untuk melengkapi salah satu syarat untuk mendaftar seminar proposal atau sidang akhir.



Gambar 3.41 Hasil Pengunduhan Buku Bimbingan dalam Bentuk PDF

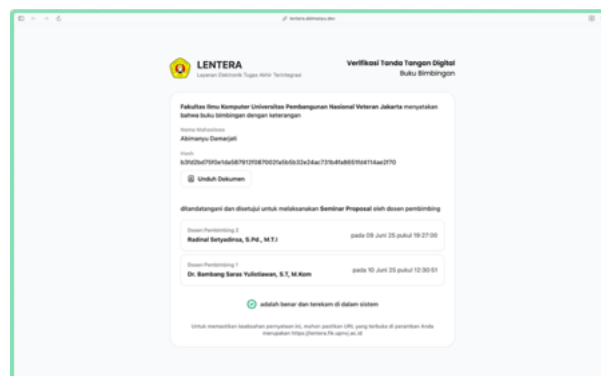
Pada bagian bawah dokumen, terdapat kode QR yang menjadi penanda bahwa dokumen tersebut sudah ditandatangani secara digital. Selain itu, terdapat informasi mengenai siapa yang melakukan tanda tangan dan kapan proses penandatanganan terjadi. Tampilan informasi

mengenai penandatanganan dokumen ditampilkan pada Gambar 3.42 di bawah.



Gambar 3.42 Tampilan Kode QR Hasil Tanda Tangan Digital

Untuk memastikan bahwa dokumen yang telah ditandatangani merupakan dokumen yang sah dan tercatat di dalam sistem, ketika pengguna memindai kode QR maka akan mengarahkan ke laman verifikasi. Laman tersebut berguna sebagai bentuk validasi bahwa dokumen yang dipindai tercatat di dalam sistem. Selain itu, pengguna yang membuka laman ini dapat mengunduh dokumen asli melalui tombol “Unduh Dokumen” sehingga dapat membandingkan terkait kesesuaian konten dokumen fisik atau digital yang diterima dengan yang tercatat di dalam sistem. Tampilan mengenai laman tersebut disajikan dalam Gambar 3.43 di bawah.



Gambar 3.43 Laman Verifikasi Dokumen yang Sudah Ditandatangani